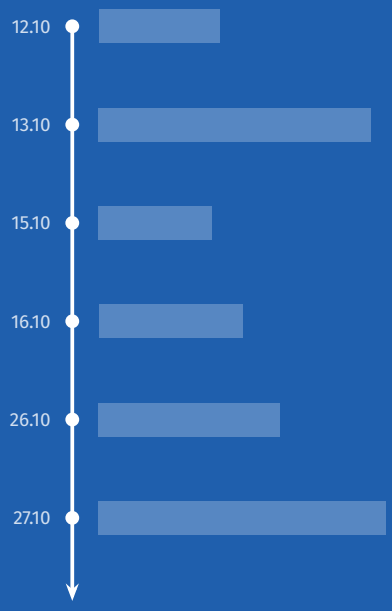
BİTİRME PROJESİ · GELİŞİM RAPORU

minGPT'den Karpathy Ailesine: Projenin Gelişimi

Bir sıralama modelinden, FPGA kartında kendi başına hikâye yazan bir model ailesine: kararlar, sayılar, hatalar ve dersler



Murat Atceken

12 – 27 Ekim 2026

Kapsam	ilk istekten (12 Ekim 23:34) bu raporun yazıldığı güne; 39 commit, 5 çalışma günü
Başlangıç	minGPT `gpt-nano`, sıralama görevi, tek modellik hızlandırıcı, PYNQ-Z2
Bugün	18 model (9 mimari, Karpathy'nin stories260K'sı, 5 TinyStories ve 3 düşük bitli model) tek mikrokodlu engine'de, Nexys A7-100T kart tasarımının tamamında 18/18 bit-exact
Doğrulama	RTL, rastgele (732 koşu, 135 180 token), 4-state, sentez sonrası netlist, zamanlama analizi
Uygulama	açık kaynak akışla yerleşim/yönlendirme: 16/18 model bitti (rapor anında)
Kaynak kod	`github.com/Muratatceken/mingpt-fpga`

Özet

Bu rapor, bitirme projesinin ilk istekten bugüne nasıl geliştiğini zaman sırasıyla anlatır. Teknik raporun (`report/rapor.pdf`) aksine burada asıl konu sonuçların kendisi değil, onlara giden yoldur: hangi kararın neden verildiği, hangi sayının hangi gün ölçüldüğü, nerede yanıldığımız ve hatayı nasıl bulduğumuz.

Proje 12 Ekim akşamı tek bir cümleyle başladı: Karpathy'nin minGPT modelini FPGA'ye uyarlamak, SystemVerilog'da yazmak ve simüle etmek. İlk gece, minGPT'nin sıralama görevindeki `gpt-nano` modeli (85 728 parametre) yalnızca integer aritmetiğiyle çalışan bir donanıma dönüştü. Donanım, Python'daki integer referansla 729 girdinin tamamında, 8 019 decode adımının her birinde bit bit aynıydı. Ertesi gün hedef büyüdü: Karpathy'nin bigram'dan nanochat'e uzanan dokuz dil modeli mimarisi eşit bütçeyle eğitildi ve hepsi tek bir mikrokodlu engine'de çalıştırıldı. Buna Karpathy'nin kendi eğittiği stories260K hikâye modeli de eklendi.

İki hafta sonra hedef kart Nexys A7-100T oldu ve önemli bir karar verildi: model, bilgisayarın sürdüğü bir hızlandırıcı olarak değil, kartın üzerinde kendi başına çalışacaktı. Üretim döngüsü, integer sampling, token'ların metne çevrilmesi ve VGA ekranı FPGA'ye girdi. Hocanın önerdiği LittleBit ve Larq yöntemleri Mac'te QAT ile denendi; donanıma 1 bitlik weight'lerle çalışan, çarpıcısız bir matris-vektör birimi eklendi. Doğrulama rastgele testler, 4-state simülasyon ve sentez sonrası netlist simülasyonuyla sertleştirildi; bu testler iki RTL hatası ve üç ayrı araç hatası buldu. Son olarak Vivado olmadan, Mac'te açık kaynak araçlarla yerleşim, yönlendirme, zamanlama analizi ve bitstream üretimi yapıldı. Fiziksel kart kullanılmayacağı için son doğrulama Mac'teki simülasyon ve zamanlama analizine dayanır.

Bugünkü durum

- **Modeller:** 18 model Nexys A7-100T kart tasarımının tamamında, Python'daki integer referansla token token, hash ve ekran karakteri olarak aynı (18/18).
- **Doğrulama:** RTL simülasyonu, 732 rastgele koşu (135 180 token), 16 yönlendirilmiş köşe durumu, X kontrollü 4-state simülasyon ve gerçek üst modülün sentez sonrası netlist simülasyonu.
- **Zamanlama:** açık kaynak akışla 16/18 model yerleştirildi; ölçülen Fmax 23,8–71,3 MHz. Büyük modeller 50 MHz'i tutmuyor, 25 MHz'te tutuyor.
- **Açık işler:** engine'in tek cycle'lık eleman bazlı yolunu pipeline'a bölerek 50 MHz; 18 modelin netlist simülasyonunu yerleştirilen netlist'le tekrarlamak; minGPT'nin sıralama modelini 19. model olarak kart tasarımına eklemek.

Her evre aynı düzende anlatılır: *neden* (karar ve motivasyon), *ne yapıldı*, *sorunlar ve çözümleri*, *çıkan ders*. Tablolardaki sayılar `report/gelisim/veri.py` ile depodaki sonuç dosyalarından üretilir; metindeki tarihli sayılar o günün commit mesajlarından ve belgelerinden alınmıştır.

İçindekiler

Özet i

1. Projeye bakış 1
1.1. Tek cümleden bir model ailesine . . . 1
1.2. Evreler . . . 1
1.3. Mimarinin evrimi . . . 2
1.4. Değişmeyen tek kural: bit-exact . . . 2

Kısım I Tek Model, Tek Kart: minGPT ve PYNQ-Z2 (12–13 Ekim) 4

2. Evre 1: minGPT'den bit-exact bir donanıma 4

3. Evre 2: Anlatmak ve ölçmek 6

4. Evre 3: Karpathy ailesi ve mikrokodlu engine 8

Kısım II Anlatmak ve Anlamak: Hoca İçin Sunum (15–16 Ekim) 12

5. Evre 4: Hoca için sunum 12

Kısım III Kartta Kendi Başına: Nexys A7-100T Dönemi (26–27 Ekim) 15

6. Evre 5: Nexys A7-100T, model kartta tek başına 15

7. Evre 6: Düşük bitli weight'ler, Larq ve LittleBit 17

8. Evre 7: Doğrulamanın sertleştirilmesi 21

9. Evre 8: Mac'te açık kaynak yerleşim, yönlendirme ve zamanlama 23

10. Evre 9: Karar, fiziksel kart yok 26

Kısım IV Bugün ve Sonrası 28

11. Bugünkü durum 28
11.1. Kart tasarımındaki modeller . . . 28
11.2. Ne bitti, ne kaldı? . . . 28
11.3. Sonraki adımlar . . . 28
11.4. Geriye bakış . . . 29

A. Commit geçmişi 30

B. Araçlar ve sürümler 32

C. Nerede ne var? 32

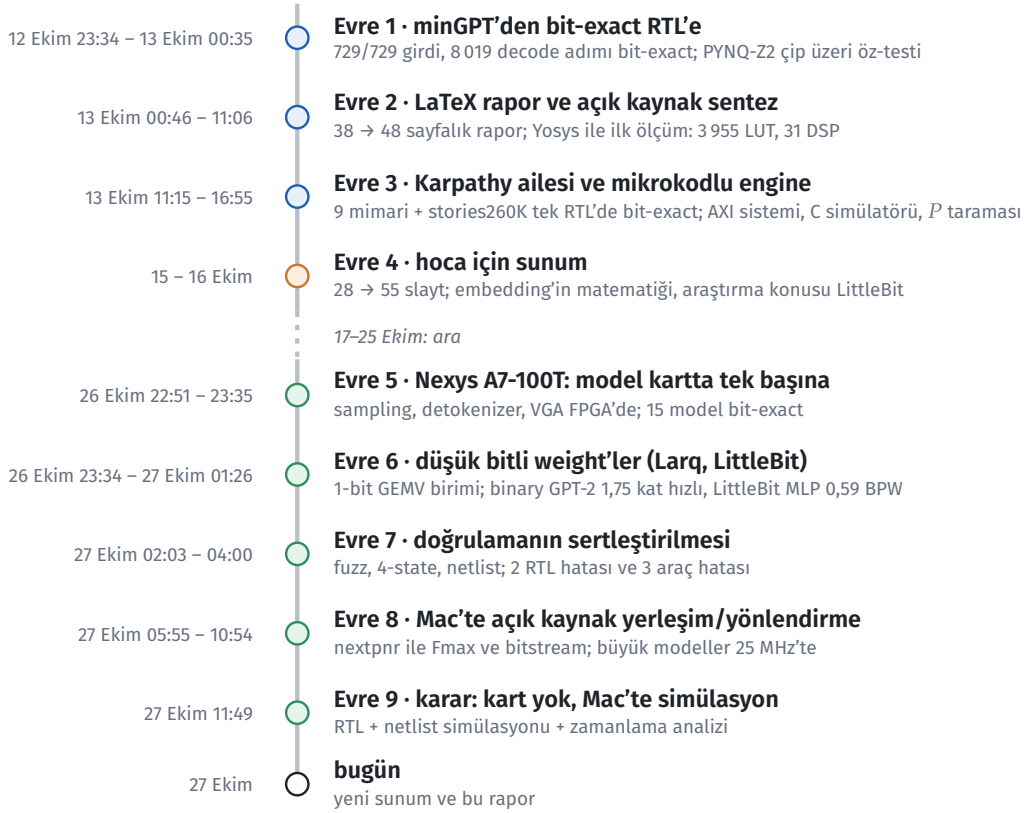
1 Projeye bakış

1.1 Tek cümleden bir model ailesine

Projenin ilk isteği, 12 Ekim akşamı yazılan şu cümleydi:

> *"şimdi ben bitirme projesi olarak https://github.com/karpathy/minGPT.git ile bunu fpga'ye uyarlamak istiyorum. bunun üzerine teorik sağlam bir şekilde çalışıp system verilogda yazıp vivadoda simüle etmek istiyorum"* 12 Ekim, 23:34

On beş gün sonra aynı depo, Karpathy'nin dokuz dil modeli mimarisini ve hazır eğitilmiş bir hikâye modelini tek bir donanımda çalıştırıyor. Bu donanım bir FPGA kartında, bilgisayara bağlı olmadan hikâye yazıyor. Aradaki yol düz bir çizgi değildi: hedef kart değişti, mimari iki kez baştan kuruldu, bir öneri kullanıcı tarafından reddedildi ve doğrulama sırasında kullandığımız araçların kendisinde hatalar bulundu. Şekil 1 bu yolu gün gün gösterir.



Şekil 1. Projenin zaman çizelgesi. 39 commit 5 farklı güne yayılıyor; 13 Ekim tek başına en yoğun gün. 17–25 Ekim arasında çalışma yapılmadı. Saatler yerel saattir (UTC+3).

1.2 Evreler

Tablo 1, raporun geri kalanının haritasıdır. Her satır bir evre ve o evrenin cevapladığı soru.

Tablo 1. Projenin evreleri.

#	Tarih	Evre	Cevaplanan soru
1	12–13 Ekim gece	minGPT'den RTL'e (§2)	Bir transformer yalnız integer aritmetikle, bit bit doğru olarak donanıma dönüşebilir mi?
2	13 Ekim sabah	Rapor ve açık kaynak sentez (§3)	Yaptığımızı anlatabiliyor ve Vivado olmadan ölçebiliyor muyuz?
3	13 Ekim gün	Karpathy ailesi ve mikrokodlu engine (§4)	Dokuz farklı mimari tek bir donanımda çalışabilir mi; aralarındaki fark neye mal olur?
4	15–16 Ekim	Sunum, embedding ve LittleBit (§5)	Yaptığımızı gerçekten anladık mı ve bir başkasına anlatabiliyor muyuz?
5	26 Ekim	Nexys A7-100T: kart tek başına (§6)	Model, bilgisayarsız, kartın kendisinde çalışabilir mi?
6	26–27 Ekim gece	Düşük bitli weight'ler (§7)	LittleBit ve Larq bu ölçekte işe yarar mı; donanıma ne girmeli?
7	27 Ekim gece	Doğrulamanın sertleştirilmesi (§8)	Testlerimiz gerçekten her hatayı yakalıyor mu?
8	27 Ekim sabah	Mac'te açık kaynak uygulama (§9)	Devre çipte kaç MHz'te çalışır?
9	27 Ekim	Karar: kart yok, simülasyon (§10)	Fiziksel kart olmadan doğrulama nasıl tamamlanır?
10	bugün	Durum ve sonraki adımlar (§11)	Ne bitti, ne kaldı?

1.3 Mimarinin evrimi

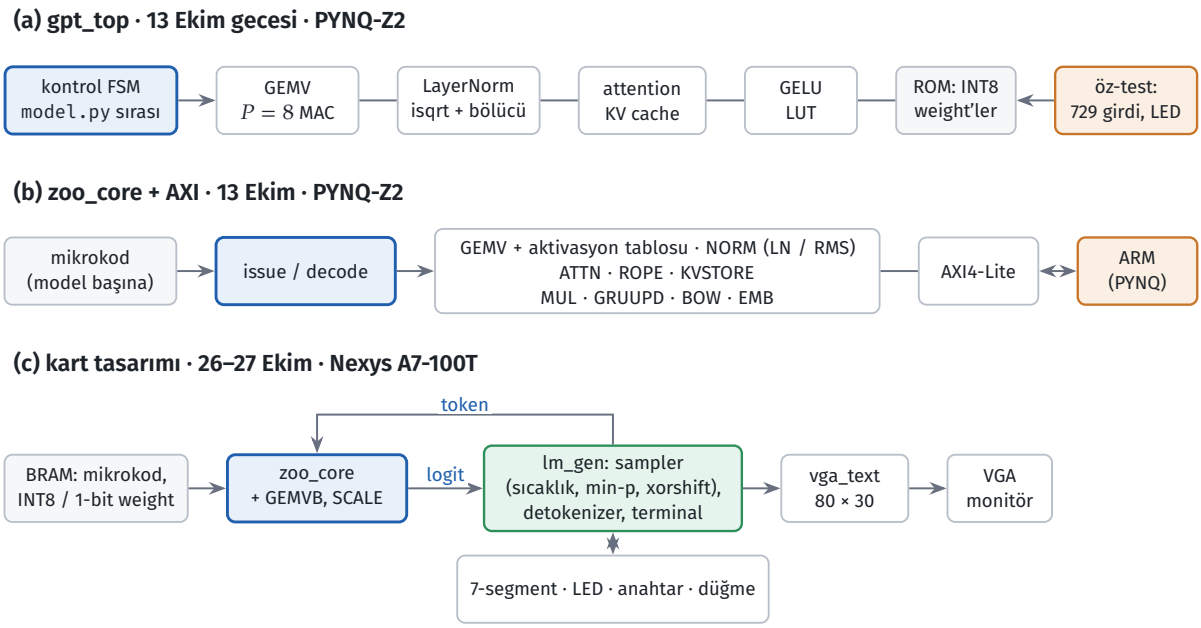
Donanım üç kez şekil değiştirdi (Şekil 2). İlk tasarım minGPT'nin `forward()` fonksiyonunu adım adım izleyen sabit bir durum makinesiydi. İkincisi, modele özgü her şeyi bir mikrokod programına ve bellek içeriklerine taşıyan genel bir engine oldu; bir ARM işlemci onu AXI üzerinden sürecekti. Üçüncüsünde işlemci tamamen çıktı: üretim döngüsü, sampling ve ekran FPGA'nin içine girdi.

1.4 Değişmeyen tek kural: bit-exact

Bütün evreler boyunca bir kural hiç değişmedi: donanımın her çıktısı, Python'da yazılmış integer referans modelin çıktısıyla *aynı* olmalı. Yaklaşık değil, aynı. Bunun sebebi basit. Donanımın yaptığı hesap tamamen integer'dır ve Python'daki referans da aynı integer hesabı yapar. İki tarafın aynı girdiden farklı bir sonuç üretmesinin tek açıklaması bir hatadır. Bu kural testleri çok güçlü kılar: tek bir token'ın, tek bir logit'in ya da ekrandaki tek bir karakterin farklı çıkması alarm demektir. Raporun ilerleyen bölümlerinde bulunan her hata bu kural sayesinde bulundu.

Float model ile integer model arasında ise bit-exact bir eşitlik beklenemez; oradaki fark *kalite* olarak, test kaybıyla (nat/token) ölçülür. Bu iki ayrı soruyu karıştırmamak raporun her yerinde önemlidir:

- **Doğru mu?** Donanım ↔ integer referans: bit-exact, == ile karşılaştırılır.
- **İyi mi?** Integer model ↔ float model: test kaybındaki artış, örneğin gpt2'de 2,000'den 2,002'ye.



Şekil 2. Donanımın üç hâli. (a) 13 Ekim gecesi: tek modele özel sabit FSM. (b) 13 Ekim: mikrokodlu ortak engine ve ARM'den AXI4-Lite ile sürülen sistem. (c) 26 Ekim'den bu yana: model kartta kendi başına; engine'e 1 bitlik weight birimi (GEMVB) ve SCALE eklendi.

KISIM I

Tek Model, Tek Kart: minGPT ve PYNQ-Z2 (12–13 Ekim)

2 Evre 1: minGPT'den bit-exact bir donanıma

12 Ekim 23:34 – 13 Ekim 00:35 · commit'ler `088c227`, `4e77f63`

Neden?

İlk istek açıktı: minGPT'yi FPGA'ye taşımak ve bunu *teorik olarak sağlam* biçimde yapmak. minGPT'nin depo içindeki en küçük örneği, `gpt-nano` yapılandırmasıyla bir sıralama görevidir: 0, 1 ve 2 rakamlarından oluşan 6 uzunluğunda bir diziyi sıralamak. Bu görev bir başlangıç için idealdi:

- **Bütün girdiler sayılabilir:** $3^6 = 729$ olası dizi var. Donanımı rastgele bir alt kümede değil, girdilerin *tamamında* test etmek mümkün.
- **Doğruluk tartışmasız ölçülür:** çıktı ya sıralıdır ya değildir.
- **Model küçük:** 3 katman, 3 attention head, $C = 48$, toplam 85 728 parametre. INT8 weight'ler yaklaşık 83 KB tutar ve çipin içine rahatça sığar.

Ne yapıldı?

İş üç katmana bölündü ve her katman bir öncekine karşı doğrulandı.

1. Matematiksel olarak eşdeğer dönüşümler. Donanımın işini azaltmak için float model önce *aynı sonucu veren* daha basit bir forma getirildi. LayerNorm'un γ ve β parametreleri sonraki Linear katmana katlandı: $W(\gamma \odot \hat{z} + \beta) + b = (W \operatorname{diag}(\gamma))\,\hat{z} + (W\beta + b)$. Attention'daki $1/\sqrt{d_h} = 1/4$ çarpanı da W_q'ya katlandı. Katlanmış float64 model, orijinal minGPT ile 729 dizinin tamamında 10^{-4} toleransla aynı logit'leri üretti.

2. Yalnızca integer bir referans model. Weight'ler ve activation'lar INT8'e, residual akış INT16'ya indirildi. Bunun kalbi *requantization*: bir matris-vektör çarpımının INT32 toplamını yeniden INT8'e indiren işlem.

Sezgi ve sayısal örnek: requant. Gerçek dünyada çıkış $y = M \cdot acc$ olmalı; burada acc integer toplam, $M = S_w S_x / S_y$ ise üç scale'in oranı, yani ondalıklı bir sayı. FPGA'de ondalıklı sayıyla çarpmak pahalı olduğu için M bir integer çarpan ve bir kaydırma olarak yazılır: $M \approx m_0 \cdot 2^{-sh}$, $m_0 \in [2^{15}, 2^{16})$.
Örnek: $M = 0{,}0123$ olsun. $0{,}0123 \cdot 2^{22} = 51\,589{,}9$ olduğundan $m_0 = 51\,590$, $sh = 22$ seçilir. $acc = 4\,321$ için

$$q_y = (4\,321 \cdot 51\,590 + 2^{21}) \gg 22 = 225\,017\,542 \gg 22 = 53,$$

gerçek değer ise $4\,321 \cdot 0{,}0123 = 53{,}148$; yuvarlanmış hâli de 53. Yalnızca bir çarpma, bir toplama ve bir kaydırma ile ondalıklı çarpmanın sonucu elde edildi. m_0'ın 16 bit olması, bağıl hatayı 2^{-16}'nın altında tutar.

LayerNorm, softmax ve GELU için de yalnız toplama, çarpma, kaydırma, taban bölme ve taban karekök kullanan algoritmalar türetildi. Her biri için taşmanın olmayacağı ispatlandı. En sevdiğimiz örnek LayerNorm çıkışıdır: Samuelson eşitsizliğine göre N elemanlı her vektörde $|x_i - \mu| \le \sigma\sqrt{N-1}$ olduğundan normalleştirilmiş değer $\sqrt{47} \approx 6{,}86$'yı geçemez. Çıkış formatı Q3.4'ün tavanı 7,94 olduğu için LN çıkışı *hiçbir girdide* doymaz. Bu, veriden bağımsız bir garantidir; kalibrasyon gerektirmez.

3. RTL. SystemVerilog'da yazılan ilk donanım (`gpt_top`) minGPT'nin `forward()` fonksiyonunu satır satır izleyen bir durum makinesiydi: embedding → LN → q, k, v → attention → projection → LN → MLP → ...→ lm_head. Birimler:

- 8 hatlı, output-stationary bir GEMV ve kanal başına requant;
- integer LayerNorm (32 cycle'lık sıralı karekök ve 48 cycle'lık sıralı bölücü);
- KV cache'li causal attention ve tablo tabanlı softmax;
- GELU tablosu ve akış hâlinde argmax.

Sonuçlar

Kart: PYNQ-Z2. Hemen ardından gelen istek "buna uygun bir kart seç sen git de kur" oldu. PYNQ-Z2 (Zynq-7020) seçildi: tasarım BRAM'in yaklaşık %17'sini kullanıyordu, Vivado'nun ücretsiz sürümü çipi destekliyordu ve kartın ARM işlemcisi ileride hız karşılaştırması için kullanılabilirdi. Karta bir *çip üzeri öz-test* eklendi: tasarım 729 sıralama problemini kendi içinde

Tablo 2. İlk donanımın sonuçları (13 Ekim, Verilator 5.052).

Ölçü	Değer
Görev doğruluğu	float, integer ve RTL: 729/729
Bit-exact	729 dizi × 11 decode adımı = 8 019 adımın her birinde token, logit'ler ve 48 residual değeri aynı
Sayısal sadakat	integer modelin logit SQNR değeri 41,5 dB; float modelle token uyumu %100
Hız	14 544 cycle/token; 100 MHz'de 145 µs; GEMV verimi %98,3
Bellek	83 KB INT8 weight, 3 KB KV cache; dış bellek yok

üretiyor, çözüyor ve sonucu kontrol ediyor; LED yeşilse 729/729 doğru. Bir anahtar çıktıları bilerek bozuyor; o durumda LED kırmızı olmalı. Simülasyonda normal modda 729/729 geçti, hata enjeksiyonunda 729/729 kaldı.

Sorunlar ve çözümleri

- **Testbench'in kendisi doğru mu?** "hep PASS veren" bir testbench'ten korunmak için referans dosyasına bilerek iki hata eklendi (bir residual değer +1, bir token değişik). Testbench ikisini de yakaladı. Bu *negatif test* sonraki her evrede tekrarlandı.
- **Bir SystemVerilog tuzağı:** `initial` bloğundaki bir döngüde ilk değerle tanımlanan değişkenler varsayılan olarak *static*'tir; ilk değer yalnız bir kez atanır. Rastgele birim testleri bu yüzden 2 000 kez aynı girdiyi deniyordu. Değişkenler `automatic` yapıldı.
- **Resmî pin dosyasına ulaşılamadı:** PYNQ-Z2'nin resmî XDC bağlantısı 404 verdi; pinler iki bağımsız kaynakla karşılaştırıldı.

Çıkan ders. Donanımın spesifikasyonu, Python'daki integer modelin kendisidir. Python'un integer'ları sınırsız hassasiyetli olduğu için RTL'deki her taşma ya da yanlış bit genişliği bir uyumsuzluk olarak ortaya çıkar; karşılaştırmada tolerans yoktur. Bu fikir, projenin geri kalanının omurgası oldu.

3 Evre 2: Anlatmak ve ölçmek

13 Ekim 00:46 – 11:06 · commit'ler `9fc2dc5`, `768ff4e`, `2b50611`

Neden?

Çalışan bir donanım tek başına bir tez değildir; ne yapıldığının ve neden doğru olduğunun anlatılması gerekir. Kullanıcının isteği iki adımdı:

> *"biz şimdi buraya çok detaylı bir min gpt nedir nasıl çalışır detaylı bir teknik ve algoritmik matematiksel anlatımı gerekiyor devamında da bunun fpga implementasyonunu çok teknik ve detaylı bir şekilde anlatmamız gerekiyor"* 13 Ekim, 00:46

> *"raporu pdf olarak hazırla şemalarını çiz çok güzel gözüksün"* 13 Ekim, 02:35

İkinci bir eksik de ölçümdü. O ana kadar kaynak kullanımı yalnızca tahmindi, çünkü Vivado macOS'ta çalışmıyor.

Ne yapıldı?

LaTeX teknik rapor (`9fc2dc5`, 09:59). 38 sayfalık, XeLaTeX ile yazılmış bir rapor: minGPT'nin matematiği, KV cache, ispatlarıyla quantization teorisi, donanım mimarisi, performans modeli, doğrulama ve kart entegrasyonu. 16 TikZ/pgfplots şeklindeki grafikler gerçek model verisinden üretildi (`sw/report_data.py`). Bu raporun kullandığı stil dosyası da o gün yazıldı.

Yön tartışması (10:38). Kullanıcı "şimdi sırada ne var?" diye sordu. O günkü değerlendirme, bir jürinin soracağı üç soruya dayanıyordu:

1. *Kartta çalıştı mı?* Her şey yalnızca simülasyondaydı.
2. *Neye göre hızlı?* 145 µs/token tek başına bir anlam taşımıyordu.
3. *Sıralama görevi bir oyuncak değil mi?* %100 doğruluk, quantization'ın zorlandığı bir durumu göstermiyordu.

Önerilen yol kartta çalıştırma, ARM ile karşılaştırma ve metin üreten bir model idi. Kullanıcı ise daha geniş bir hedef koydu (§4):

> *"karpathynin diğer bu tarz modelleri de var nanogpt gibi. onların hepsine bakıp yine hepsini implemente edip hepsini karşılaştırdığım bir hale çevirmek istiyorum işi. rapordaki matematiksel anlatımı beğendim. bir de görsel olarak sağlam bir anlatım ekleyelim."* 13 Ekim, 10:43

Raporun II. kısmı (`768ff4e`, 10:57). Karpathy'nin depoları tek tek indirilip mimariler doğrudan koddan incelendi. Rapor 48 sayfaya çıktı. Eklenen dört ispatlı sınır, sonraki evrede donanımın bit genişliklerini belirledi:

- RNN/GRU gizli durumu $|h| \leq 1$: kalibrasyonsuz Q0.7 yeterli;
- RMSNorm çıkışı $\leq \sqrt{N}$;
- RoPE'de skor yalnız göreli konuma bağlı;
- QK-norm'da skor $\leq c^2\sqrt{d_h}$: softmax ölçeği analitik olarak sabitlenebilir.

Görsel anlatım için eğitilmiş modelin 9 head'inin gerçek attention haritaları çizildi. Bir gözlem de çıktı: ilk katman, çıktı adımlarında attention'ın %84–92'sini girdi rakamlarına veriyor; düzgün dağılımda bu oran %77,5 olurdu. Bu, modelin rakamları sayarak sıraladığı hipotezine uyuyor; raporda kanıt olarak değil hipotez olarak geçti.

Mac'te açık kaynak sentez (`2b50611`, 11:06). Kullanıcı "vivadoyu veya benzer bir simülatörü buraya kuramaz mıyız?" diye sordu. Vivado'nun macOS'ta doğrudan çalışmadığı, Docker + Rosetta ile çalıştırma yolunun ise dört engeli olduğu görüldü: macOS sürümü (yöntem macOS 15'te denenmişti, bu Mac 27'de), yaklaşık 20 GB'lık kurulum için yetersiz disk (57 GB boş), AMD hesabıyla giriş ve sınırlı JTAG desteği. Onun yerine sv2v + Yosys kuruldu. Yosys `synth_xilinx`, Xilinx 7-serisi için gerçek bir sentez yapar ve 26 saniyede çalıştı:

Tablo 3. `gpt_top`'un ilk ölçülen kaynakları (Yosys, xc7z020'ye göre).

Kaynak	Kullanılan	Pay
LUT	3 955	%7,4
FF	1 567	%1,5
BRAM (RAMB36 eşdeğeri)	23	%16,4
DSP48E1	31	%14,1

Raporun "tahmin" yazan yerleri bu ölçümlerle değiştirildi. Ölçülemeyen tek şey zamanlamaydı (Fmax); Yosys bunu yapmıyor. Bu boşluk 14 gün sonra, §9'de kapandı.

> *"her şeyi bu bilgisayarda yap en uygun neyse onla çalış işleri bitir eğitimi yap testleri yap"* 13 Ekim, 11:15

Bu cümle projenin çalışma biçimini belirledi: eğitim, simülasyon, sentez ve belgeler Mac'te; yalnızca Mac'te yapılamayanlar kullanıcıya not olarak bırakılacaktı.

Çıkan ders. Ölçülmemiş bir sayı tahmindir ve öyle yazılmalıdır. Tahminler ölçümle değiştikçe (ör. DSP için 16–30 tahmini yerine ölçülen 31) raporun güvenilirliği arttı. Bir aracın eksik olduğu yerde (Vivado) durmak yerine o aracın yaptığı işin ölçülebilen kısmını başka bir araçla yapmak, ilerlemeyi sürdürdü.

4 Evre 3: Karpathy ailesi ve mikrokodlu engine

13 Ekim 11:15 – 16:55 · 14 commit, `adf3075` … `6bb4ae9`

Neden?

Kullanıcının hedefi tek modelden bir aileye çıktı: Karpathy'nin modellerinin hepsini implemente etmek ve karşılaştırmak. Survey, dil modeli mimarisinin 2003'ten 2026'ya evrimini izleyen dokuz mimari çıkardı:

- **attention öncesi (makemore):** Bigram, MLP (Bengio 2003), RNN, GRU, BoW;
- **GPT-2:** minGPT ve nanoGPT inference'ta aynı işlemleri yaptığı için tek mimari;
- **microgpt** (Şubat 2026): RMSNorm (parametresiz), ReLU, bias yok;
- **Llama 2** (llama2.c): RMSNorm, RoPE, SwiGLU, GQA;
- **nanochat** (çekirdek mimari): RoPE, QK-norm, ReLU², GQA.

llm.c (nanoGPT ile aynı mimari), char-rnn (makemore'un RNN/GRU'su kapsıyor) ve micrograd (dil modeli değil) dışarıda kaldı.

Asıl tasarım kararı şuydu: *dokuz ayrı hızlandırıcı yazmak yerine tek bir engine*. Sabit durum makinesinin yerini, Python derleyicisinin her model için ürettiği küçük bir komut dizisi (mikrokod) aldı. Modele özgü olan her şey (program, weight'ler, requant parametreleri, tablolar) bellek içeriği oldu; RTL bütün modellerde aynı kaldı. Bu, karşılaştırmayı adil kılar: farklı modellerin cycle sayısı farkı, tasarımcının bir modele daha çok emek vermesinden değil mimarinin kendisinden gelir.

Ne yapıldı?

Dokuz model, tek RTL (`adf3075`, 12:05). Dokuz model `names.txt` üzerinde yaklaşık 50 bin parametrelik eşit bir bütçeyle eğitildi. Bir IR derleyicisi her modeli ortak işlemlere indirdi; quantization ve bit-exact bir integer yorumlayıcı (`IntRunner`) eklendi; `rtl/zoo/zoo_core.sv` mikrokodlu engine olarak yazıldı. Dokuz modelin hepsi Verilator'da bit-exact çıktı (354 adım × 27 logit). Tablo 4 o günün sonuçlarını gösterir.

Tablo 4. Karpathy ailesi, PYNQ dönemi engine'inde. Kayıplar names test kümesinde nat/token; cycle/token tek bitstream'lik genel engine'de, $P = 8$. stories260K için ilk ve son pozisyonun cycle'ı verilmiştir.

Model	Parametre	Float kayıp	INT8 kayıp	Δ	cycle/token
`bigram`	729	2,451	2,451	−0,0000	36
`mlp`	50 459	1,986	1,987	0,0011	6 632
`rnn`	49 347	1,964	1,967	0,0032	6 374
`gru`	48 823	1,956	1,956	0,0003	6 515
`bow`	49 979	2,045	2,044	−0,0003	6 278
`gpt2`	53 120	2,000	2,002	0,0017	8 620
`microgpt`	51 392	1,971	1,974	0,0032	8 657
`llama`	50 304	2,003	2,004	0,0013	9 396
`nanochat`	46 784	1,969	1,975	0,0055	10 711
`stories260k`	260 K	1,400	1,422	0,0217	38 652 … 79 532

Hazır eğitilmiş bir hikâye modeli (`2aef51f`, 13:01). Karpathy'nin llama2.c deposundaki stories260K modeli (TinyStories, 260 bin parametre) aynı akıştan geçirildi: `.bin` yükleyici, llama2.c'nin `run.c` tokenizer'ı ve INT8. RTL 1 024 adımda bit-exact çıktı ve referansla token token aynı 200 tokenlık bir hikâye üretti. Bu, projede *Karpathy'nin kendi eğittiği weight'lerle* çalışan tek modeldir.

Quantization incelemesi. stories260K'da INT8 kaybı ilk denemede 0,052 nat/token'dı. Hata kaynağını bulmak için *fake quantization* ile tek tek ablasyon yapıldı: weight'leri, activation'ları, normları ve residual'ı ayrı ayrı quantize etmek. Kaybın weight'lerden değil activation'lardan geldiği görüldü. Aktivasyon scale'leri en büyük değer yerine %99,99 yüzdelik değerle seçilince ve NORM çıkış ölçeği kalibre edilince fark 0,052'den 0,022'ye indi.

RTL iyileştirmeleri. Sınırlı bölümlü bir bölücü (`zdiv`) NORM'u 64'ten 41 cycle'a, softmax'ı 34'ten 18 cycle'a indirdi. İki sentez parametresi eklendi:

- `SPECIALIZE`: modelin kullanmadığı birimler sentezlenmez.
- `LOADABLE`: bellekler işlemciden yüklenebilir yapılır. Yosys, içeriği sabit ROM'ları sentez sırasında mantığa katlayıp yanıltıcı derecede küçük sonuçlar veriyordu; yüklenebilir bellekte bunu yapamaz.

"Paper kalitesinde" plan (13:30). Kullanıcı "ben profesyonle paper kalitesinde bir iş istiyorum" dedi. Plan, çalışmanın iddiasını tek cümleye indirdi: ailenin her mimari yeniliğinin (attention, RMSNorm, RoPE, SwiGLU, GQA, QK-norm) donanım maliyetini ve kaliteye katkısını aynı engine'de, kontrollü olarak ölçmek. Dört araştırma sorusu ve altı iş paketi tanımlandı: çok tohumlu eğitim ve öğrenme oranı taraması, GPT-2'den Llama'ya tek tek özellik değiştiren bir *özellik merdiveni*, ölçek taraması ve TinyStories'te ikinci bir deney hattı (Hat B). Kullanıcının bir sonraki cümlesi işin bölüşümünü belirledi:

> *"sen bu bilgisayarda yapılacak her şeyi yap geri kalanı bana bırak neyapmam gerektiğini detaylı bir not olarak bırak"* 13 Ekim, 13:39

Bu not `docs/SENIN_YAPACAKLARIN.md` oldu: Vivado, kart, güç ölçümü ve makale başvurusu için adım adım talimatlar.

Altyapı (14:26 – 15:47). Aynı öğleden sonra eklenenler:

- **C ISA simülatörü** (`sw/c/zoo_run.c`): mikrokodu C'de yorumlar, 10 modelde bit-exact; CPU'da temel çizgi.
- **Rastgele model testi** (`fuzz.py`): rastgele mimari, boyut ve weight'lerle RTL–referans karşılaştırması; 60 vaka, 1,26 milyon logit, 0 hata.
- **Aritmetik birimlerin testi:** `zdiv` $W = 12$'de bütün girdilerle (tam tarama), ayrıca 36,4 milyon rastgele vaka; karekök için rastgele ve sınır vakaları.
- **Tek bitstream engine'i:** on modelin en büyük boyutlarıyla ($V = 512$, $T = 256$, 5 katman) tek bir engine; her model çalışma anında yüklenir. Yosys'e göre BRAM önce 132 idi (çipin %94'ü); BoW tablosu 16 bitlik iki kelimeye bölününce 114 oldu (xc7z020'nin %81'i).
- **AXI4-Lite sistemi ve PYNQ sürücüsü:** adım, yükleme, logit penceresi, donanım argmax'ı ve cycle sayacı için yazmaç haritası; testbench kart sürücüsünün yaptığını yapar, 10 model ve 200 tokenlık hikâye bit-exact.
- **Kapalı biçimli cycle modeli** (`perf_model.py`): mikrokoddan her pozisyonun cycle sayısını hesaplar; RTL ile cycle'ı cycle'ına aynı.
- **CI:** her push'ta GitHub Actions'ta Verilator regresyonu; Dockerfile; IEEE makale iskeleti.

Tasarım uzayı: P taraması (`6bb4ae9`, 16:55). Attention'ın head genişliği ($d_h = 8$) GEMV hat sayısından (P) ayrıldı; böylece $P = 8, 16, 32$ denenebildi. 6 model × 3 P = 18 noktanın hepsi bit-exact çıktı ve cycle modeli her noktada ölçümle birebir tuttu (Tablo 5).

Tablo 5. GEMV hat sayısı P'nin etkisi (cycle/token). GEMV'deki hızlanma ile toplam hızlanma arasındaki fark Amdahl yasasıdır: stories260K'da zamanın büyük kısmı attention'da geçtiği için GEMV'yi hızlandırmak toplamı daha az etkiler.

Model	$P = 8$	$P = 16$	$P = 32$	GEMV, 8→32	Toplam, 8→32
`mlp`	6 632	3 480	2 056	3,56×	3,23×
`gru`	6 515	3 683	2 315	3,07×	2,81×
`gpt2`	8 620	5 684	4 540	2,62×	1,90×
`llama`	9 396	6 492	5 548	2,37×	1,69×
`nanochat`	10 711	8 031	7 143	2,42×	1,50×
`stories260k`	59 092	43 012	35 724	3,37×	1,65×

Sorunlar ve çözümleri

- **Yosys'in ROM katlaması:** sabit içerikli bellekler mantığa katlanınca kaynaklar olduğundan küçük görünüyordu → `LOADABLE=1` ile sentez.
- **Tek bitstream engine'i çipi neredeyse dolduruyordu** (132 BRAM, %94) → BoW tablosunun 16 bitlik kelimelere bölünmesiyle 114.
- **AXI testbench'inde yarış durumu:** el sıkışmaları Verilator sürümüne göre farklı cycle'da görülüyordu → sinyaller düşen kenarda sürülüp örneklendi.
- **Netlist simülasyonu sonuçsuz kaldı:** Yosys'in hücre modellerinde X değerleri yayılıyordu. Bu sorun 14 gün sonra, kendi BRAM modelimiz yazılarak çözüldü (§8).

Çıkan ders. Genellik, adil karşılaştırmanın ön koşuludur. Tek bir engine, "bu model neden daha yavaş?" sorusunu tasarımcıya değil mimariye sorar. İkinci ders: bir hızlandırıcının hızlandırmadığı kısım sonunda darboğaz olur ($P = 32$'de stories260K).

KISIM II

Anlatmak ve Anlamak: Hoca İçin Sunum (15–16 Ekim)

5 Evre 4: Hoca için sunum

15 Ekim 11:01 – 16 Ekim 14:21 · 6 commit, `08523c8` … `81b3641`

Neden?

İlk hafta kod ve ölçümle geçti; sıra bunları bir hocaya anlatmaya geldi. Kullanıcının isteği yalnızca bir sonuç sunumu değildi, aynı zamanda bir sınavdı:

> *"şu ana kadar neler yaptık, teorik zeminimiz ne, en önemlisi mingpt'nin matematiksel olarak adım adım anlatılması ne nedir, bunu kod blokları ile de bu bunu yapıyor gibisinden göstermek.* […] *hem bir şu ana kadar neler yaptık hem de yaptığımız şeyi gerçekten anladık mı anlamadık mı diye onları da hocaya sanki o bilmiyormuş gibi anlatmamız gerekiyor."* 15 Ekim, 11:05

Ne yapıldı?

İlk taslak: 46 slayt. Bütün projeyi kapsayan bir taslak hazırlandı: teorik zemin, minGPT adım adım (her adımda formül ve onu yapan minGPT kodu yan yana), integer aritmetik, ilk donanım, model ailesi ve engine, doğrulama, ve "gerçekten anladık mı?" başlıklı beş soru ($\sqrt{d_h}$'ye neden bölüyoruz, LN neden taşmaz, softmax hileleri neden doğru, bit-exact ne demek, isimlerde GRU neden önde). Hazırlık sırasında iki bulgu da çıktı: eşit bütçede bütün modellerin matris çarpımı cycle'ı aşağı yukarı aynı (5 900–6 700); transformer'lardaki fazlalık normalizasyon ve attention'dan geliyor. $P = 32$'de stories260K'nın yalnızca 1,65 kat hızlanması da Amdahl yasasıyla tam tutuyor.

Daraltma ve iki kural. Kullanıcı sunumun dilini ve kapsamını iki cümleyle belirledi. Bu iki kural o günden beri her slaytta geçerli:

> *"biz bu tarz kavramları teknik terimleri olduğu gibi bırakalım çevirmeylim sunumda hiç bir şekilde. birde biz attentionı falanda çok temelden anlatalım nedir ne yapar gibisinden."* 15 Ekim, 12:05

> *"sunumun şu anlık sadece mingpt kısmını hazırla. bunun fpga'de uygulama potasiyellerinden bahset geri kalan kısımdan bahsetme. onu sonra yapıcaz."* 15 Ekim, 12:13

Sunum 28 slaytlık bir minGPT bölümüne indi (`08523c8`); konuşma metni slayt slayt yazıldı ve PDF'e çevrildi; 46 slaytlık taslak `sunum/v1_tam_taslak` altında saklandı.

Hocanın geri bildirimi (16 Ekim). Hoca iki şey istedi:

> *"hoca sunumda embedding kısmını matematiksel olarak daha detaylı teknik olarak tam nasıl işlediğini anlatmamızı istedi bir de araştırma konusu olarak LittleBit: Ultra Low-Bit Quantization via Latent Factorization konusunu verdi* [...] *işin matematiğinin nasıl işlediği hoca için çok önemli"* 16 Ekim, 13:05

İlk sürüm 41 slayta çıktı (`686116f`): one-hot çarpımının satır seçimine inmesi, sparse gradient, learned/sinusoidal/RoPE position embedding ve LittleBit'in (Lee vd., NeurIPS 2025) low-rank factorization, binarization, multi-scale scale'ler, Dual-SVID ve residual compensation adımları. Kullanıcının tepkisi açıktı:

> *"littlebit daha iyi açıklanmalı en salak insan bile anlamalı. ben ilk bakışta anlamıyorum. daha iyi anlatmamız şart. matematiksel olarak koru ama tam olarak ne yaptığını trickini düzgünce ltieral olarak anlatalım"* 16 Ekim, 13:30

LittleBit bölümü sezgiden matematiğe gidecek şekilde baştan kuruldu (52 slayt, `53caa4d`). Yöntemin hilesi tek cümleye indi: *matrisi sayı sayı saklamak yerine birkaç $\pm$ desen ve birkaç scale ile tarif etmek*. Matematik ise şu:

LittleBit tek formülde. Bir weight matrisi $W \in \mathbb{R}^{d_o \times d_i}$ şu şekilde yaklaşılır:

$$\hat{W} = \mathrm{diag}(h)\ \mathrm{sign}(U)\ \mathrm{diag}(\ell)\ \mathrm{sign}(V)^{\top}\ \mathrm{diag}(g), \qquad U \in \mathbb{R}^{d_o \times r},\ V \in \mathbb{R}^{d_i \times r}.$$

$\mathrm{sign}(U)$ ve $\mathrm{sign}(V)$ yalnız ± 1 içerir (1 bit), h, ℓ, g ise satır, latent ve sütun scale'leridir. Hesap $\hat{W}x$, iki adımda çarpıcısız yapılır: önce x, g ile ölçeklenir; sonra ± 1'lerle toplanır ve çıkarılır. Saklanan bit, weight başına

$$\mathrm{BPW} = \frac{r(d_o + d_i) + 16\,(d_o + d_i + r)}{d_o\, d_i}$$

kadardır (16 bitlik scale'lerle, tek yol). Bu formül, üç gün sonra küçük modellerdeki asıl bulgumuzu (§7) önceden haber veriyordu.

Bir 4×4 elle örnek, residual yolun neden işe yaradığını tek bakışta gösterdi: göreli hata SVD'de (sıkıştırmasız) %0,4; tek yollu LittleBit başlangıcında %17,7; scale'ler ve işaretler eğitilince %14,5; residual yolla %0,4.

Kavramlar. Kullanıcı sunumu kendisi anlatacağı için kavramları tek tek sordu: SVD, low-rank factorization, LoRA, quantization, residual, BPW, sign, scale. Cevaplar slayt oldu (55 slayt, `81b3641`): sekiz terimin tek cümlelik haritası; quantization'ın INT8, INT4 ve binary için sayı doğrusu ve hesaplanmış örnekleri (hata %0,15 / %2,7 / %32, en iyi $\alpha = \mathrm{ort}|w|$); LoRA ile karşılaştırma (W'ye bir düzeltme eklemek ile W'nin yerine geçmek); "residual" kelimesinin residual connection ile karıştırılmaması uyarısı. Benzetme olarak kullanılan "ses düğmesi", kullanıcının isteğiyle teknik terim olan *scale*'e bırakıldı (`f9b9055`).

Sorunlar ve çözümleri

- **Hesap değişti:** 16 Ekim'de kullanıcının hesabı değişti; eski hesaptaki sunum yeni hesaptan açılmadığı için deck yeniden oluşturuldu.
- **Acrobat'ta beyaz ekran:** PDF, Acrobat'ta açıkken üzerine yazılınca bozuk görünüyordu. O günden beri PDF'ler önce geçici bir dosyaya yazılıp sonra taşınıyor (bu rapor da öyle).
- **Kapsam:** donanım ve sonuç bölümleri bilinçli olarak sunumun dışında bırakıldı ("onu sonra yapıcaz").

Çıkan ders. Bir şeyi anlatmaya çalışmak, onu gerçekten anlayıp anlamadığımızı gösterir. LittleBit'in ilk anlatımı doğruydu ama anlaşılmıyordu; sezgiden başlayıp gerçek bir sayısal örnekle matematiğe gitmek, hem anlatımı hem de bizim kavrayışımızı düzeltti. Bu düzen (önce sezgi ve gerçek sayılar, sonra matematik; terimler İngilizce) bu raporda da kullanıldı.

KISIM III

Kartta Kendi Başına: Nexys A7-100T Dönemi (26–27 Ekim)

6 Evre 5: Nexys A7-100T, model kartta tek başına

26 Ekim 22:51 – 23:35 · commit d76121c

Neden?

On günlük aradan sonra proje yeni bir hedefle yeniden başladı:

> *"şimdi fpga olarak nexys A7 100t olarak ayarla bundan sonar bunun üzerinde çalışcaz bir de tam implementasyonları yap macde bir test edelim bakalım çalışıyor mu bu implemenetasyonlar sadece karpathy modellerini deniycez ama başka bir şeyi değil. biz fpgade train etmeyeceğiz fpga sadece bir inference aracı. hocamız bize yardımcı olması için littlebit makalesinden ve larq pythondan bahsetti* […] *performansa göre sen karar ver"* 26 Ekim, 22:51

Nexys A7-100T'de PYNQ-Z2'deki gibi bir ARM işlemci yok. İlk refleks, PYNQ dönemindeki düzeni korumaktı: modeli bilgisayarın bir seri port (UART) üzerinden sürdüğü bir hızlandırıcı. Bu katmanın yazılmasına başlanmışken kullanıcı durdurdu:

> *"uart yapma, ben modelin fpga üzerinde çalışmasını istiyorum"* 26 Ekim, 22:58

Gerekçe haklıydı: bilgisayarın her adımda komut gönderip sonucu okuduğu bir düzende model "bilgisayarda çalışıyor, FPGA yalnızca çarpma yapıyor" gibi görünür. Bu karar projenin en önemli mimari dönüşümlerinden biri oldu: *üretim döngüsünün tamamı FPGA'ye girdi.*

Ne yapıldı?

Kart tasarımı (`board/nexys_a7/`) dört modülden oluşur:

Tablo 6. Eski ve yeni hedef kart.

	Nexys A7-100T (xc7a100t)	**PYNQ-Z2 (xc7z020)**
LUT	63 400	53 200
Flip-flop	126 800	106 400
BRAM (36 Kb)	135 (≈ 607 KB)	140 (≈ 630 KB)
DSP48E1	240	220
İşlemci	yok	çift çekirdek ARM
Kullanılan G/Ç	VGA, 8 haneli 7-segment, 16 anahtar, 4 düğme + reset, LED'ler	—

- `nexys_a7_top`: 100 MHz osilatörden MMCM ile çekirdek clock'u (50 MHz; 25 ya da 100 seçilebilir), düğme debounce, anahtar senkronizasyonu, LED'ler.
- `lm_gen`: üretim döngüsü, sampler, detokenizer, 80 sütunluk terminal (kelime kaydırma, scroll), durum satırı ve kart üstü kontrol.
- `vga_text`: 640 × 480 @ 60 Hz metin ekranı, 80 × 30 karakter, 8 × 16 font.
- `seg7`, `bin2bcd`: 7-segment sürücü ve sıralı ikili→BCD dönüştürücü.

Weight'ler, tablolar ve mikrokod bitstream ile birlikte BRAM'e yüklenir; kart açıldığında ilk koşu kendiliğinden başlar.

Integer sampling. PYNQ döneminde token seçimi argmax'tı (greedy). Bir hikâye modelinde greedy seçim aynı cümleleri tekrarlar; gerçek üretim için olasılığa göre örnekleme gerekir. Bunun FPGA'de, kayan nokta olmadan ve Python'la bit bit aynı yapılması gerekiyordu.

Sezgi ve sayısal örnek: sampling. Softmax'ın olasılıkları $p_k \propto e^{l_k/\tau}$'dir. Üs almayı 2 tabanına çevirip en büyük logit m'yi çıkarırsak $e_k = 2^{-(m-l_k)\,c}$ olur; $c = s_{\text{logit}} \log_2 e/\tau$ sabit noktada saklanır ve üs alma attention'ın kullandığı $2^{-k/256}$ tablosuyla yapılır. Örnek: üç token'ın logit'leri $l = (120, 100, 40)$ ve $c = 0{,}05$ olsun. Üsler $(m-l)\,c = (0, 1, 4)$, ağırlıklar $e = (1;\ 0{,}5;\ 0{,}0625)$, toplam 1,5625.

- **min-p:** $e_k < e_{\max} \cdot 2^{-\text{NCUT}}$ olan token'lar atılır. NCUT = 5 için eşik $1/32 = 0{,}031$; üçüncü token (0,0625) kalır.
- **Seçim:** xorshift32 üretecinden gelen r ile eşik thr $= (r \cdot \sum e) \gg 32$ hesaplanır. $r/2^{32} = 0{,}8$ ise thr = 1,25; birikimli toplam $1 < 1{,}25 \le 1{,}5$ olduğu için ikinci token seçilir.

Gerçekte bütün değerler integer'dır; aynı algoritma `sw/zoo/board.py:pick` içinde birebir yazılıdır. Ek maliyet yaklaşık $3V$ cycle'dır: stories modellerinde yaklaşık 1 500, names modellerinde 80.

Kart üstü kontrol. Kart, ürettiği token'lardan bir hash hesaplar ($h \leftarrow 31h + \text{tok}$) ve bunu, Python'un dışa aktarma sırasında hesaplayıp bitstream'e gömdüğü değerle karşılaştırır. LED16 yeşil yanarsa kart, Python'daki integer modelle token token aynı sonucu üretmiştir. Bu, ilk günün çip üzeri öz-testinin (§2) üretici bir modele uyarlanmış hâlidir.

Mac'te test. Vivado macOS'ta çalışmadığı için kartın *bütün üst modülü* Verilator'da simüle edildi: düğmeye basılıyor, anahtarlar çevriliyor; üretilen her token, hash, kontrol LED'i ve 80 × 28'lik metin alanı karakter karakter Python referansıyla (`board.py`) karşılaştırılıyor. Commit anında 15 modelin hepsi (9 names modeli, stories260K ve TinyStories'te eğitilen 5 aile modeli) aynıydı. stories260K 50 MHz'de yaklaşık 1 000 token/s üretiyor ve BRAM'in %72'sini kullanıyordu.

Sorunlar ve çözümleri

- **Ekranda bir token kayması:** metin, üretilen token'ın bir öncekini gösteriyordu. Sebep, token tablosunun adresinin bir cycle geç (register'dan) gelmesiydi → ROM adresleri kombinasyonel yapıldı.
- **Kelimenin ortasından satır kırma:** 80. sütuna gelen kelime ikiye bölünüyordu → kelime satır sonuna sığmazsa bir sonraki satıra taşınıyor (RTL'de ayrı durumlar, Python'daki `Terminal` modelinde aynısı).
- **Sabit kodlanmış durum satırı sütunları** → şablondan bir düzenli ifadeyle türetildi; model başlığı kısaltıldı ve gerçek parametre sayısını gösterdi.
- **macOS'ta link hataları:** önce conda'nın `ld`'si seçiliyordu; sonra Xcode'un linker'ı Command Line Tools SDK'sını okuyamadı → PATH sırası ve `SDKROOT`, Xcode'un SDK'sına ayarlandı. Aynı sorun 27 Ekim'de araç zinciri derlenirken tekrar çıktı (§9).
- **BRAM sınırı:** TinyStories modelleri 512 token'la eğitildi, ama KV cache'in sığması için context 256 token'la sınırlandı.

Çıkan ders. Kullanıcının "uart yapma" itirazı, teknik bir tercihten çok projenin ne iddia ettiğiyle ilgiliydi. "FPGA'de çalışıyor" cümlesinin dürüst olması, sampling'i ve ekranı da donanıma almayı gerektiriyordu. Bu da Python referansının kapsamını büyüttü: artık yalnızca logit'ler değil, ekrandaki her karakter de spesifikasyonun parçası.

7 Evre 6: Düşük bitli weight'ler, Larq ve LittleBit

26 Ekim 23:34 – 27 Ekim 01:26 · commit'ler `8db3890`, `fc6d07a`, `7059b2e`, `76fbd65`

Neden?

Hoca iki kaynak önermişti: LittleBit makalesi ve Larq (TensorFlow/Keras için bir binary neural network kütüphanesi). Kullanıcının sorusu bunların projede nasıl kullanılacağıydı:

> *"biz burada littlebit veya larq python gibi şeyleri nasıll kullanmalıyız sence bu projede, birde bu projeyi daha da geliştirip daha sağlam hale getirebiliriz bu tarz şeylerle paralel de bunu da yapalım"* 26 Ekim, 23:34

Kural değişmedi: *FPGA'de eğitim yok*. Düşük bitli bir modeli elde etmek için gereken eğitim (QAT, quantization-aware training) Mac'te yapılır; FPGA yalnızca sonucu çalıştırır. Soru iki

parçaya ayrıldı: (1) bu yöntemler 50 bin ve 260 bin parametrelik modellerde kaliteyi ne kadar korur, (2) donanıma hangisi girmeli? Aynı anda, kullanıcının "daha sağlam hâle getirelim" isteği için doğrulamayı sertleştirme işi paralel bir dalda başladı (§8).

Ne yapıldı?

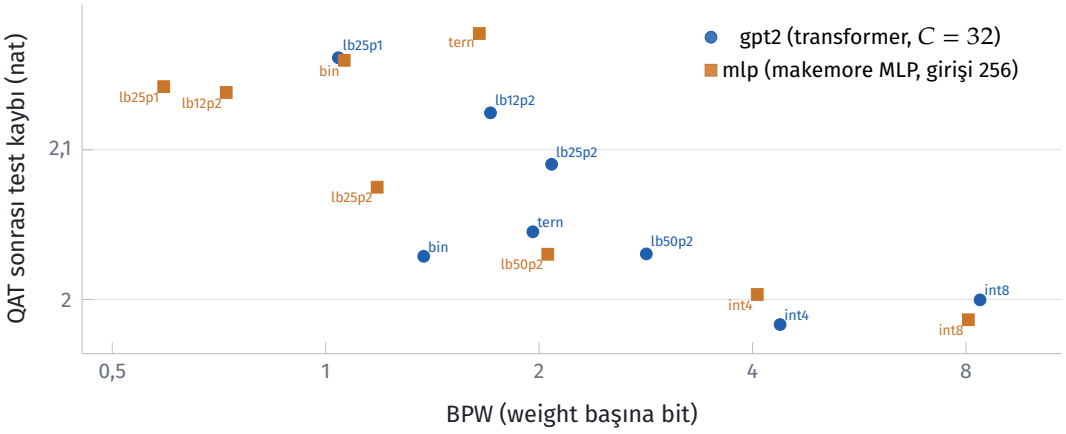
Larq'ı TensorFlow'a geçmeden kullanmak. Larq'ın quantizer'ları birkaç satırlık fonksiyonlardır: ileri yönde $\text{sign}(w)$, geri yönde kırpılmış özdeşlik (straight-through estimator). Bunlar Larq'ın belgelenen tanımlarına göre PyTorch'ta yeniden yazıldı (`sw/zoo/lowbit.py`): `SteSign`, `MagnitudeAwareSign` (Bi-Real Net), `SteTern` (ternary weight networks) ve weight clip. Böylece modeller, eğitim düzeni ve integer akış değişmedi. Gerçek `larq` paketiyle sayısal bir karşılaştırma yapılmadı; bunun için TensorFlow kurulumu gerekiyor.

LittleBit. Dual-SVID başlangıcı (SVD'den işaret ve büyüklükleri ayırarak U, V, h, g, ℓ'yi kurmak), isteğe bağlı residual yol ve faktörlerin kaydedilmesi. Eğitim: float model öğretmen, sonraki token dağılımlarına KL divergence ile knowledge distillation; Adam, cosine öğrenme oranı. Token embedding ve output head, makaledeki ve binary ağ literatüründeki gibi INT8 kaldı.

Sonuçlar: names. Sekiz names modeli × sekiz yöntem (Tablo 7, Şekil 3).

Tablo 7. QAT sonrası test kaybı (nat/token); parantez içinde BPW. `int8` eğitimsiz, diğerleri 3 000 adım QAT. `lb<r>p<k>`: rank = $\min(d)$'nin $\%r$'si, k yol.

Model	float	int8	int4	tern	bin	lb50p2	lb25p2	lb25p1	lb12p2
`bow`	2,045	2,045 (8,15)	2,049 (4,15)	2,154 (1,73)	2,122 (1,15)	2,136 (1,82)	2,186 (1,20)	2,246 (0,60)	2,239 (0,90)
`gpt2`	2,000	2,000 (8,38)	1,983 (4,38)	2,045 (1,96)	2,029 (1,38)	2,030 (2,83)	2,090 (2,08)	2,162 (1,04)	2,125 (1,71)
`gru`	1,956	1,957 (8,11)	1,965 (4,11)	2,052 (1,69)	2,028 (1,11)	1,990 (2,32)	2,030 (1,43)	2,086 (0,71)	2,088 (0,96)
`llama`	2,003	2,002 (8,42)	1,980 (4,42)	2,051 (2,00)	2,052 (1,42)	2,038 (3,29)	2,090 (2,44)	2,167 (1,22)	2,123 (2,01)
`microgpt`	1,971	1,971 (8,38)	1,966 (4,38)	2,086 (1,96)	2,059 (1,38)	2,031 (3,25)	2,088 (2,38)	2,156 (1,19)	2,128 (1,94)
`mlp`	1,986	1,986 (8,06)	2,003 (4,06)	2,178 (1,65)	2,160 (1,06)	2,030 (2,06)	2,075 (1,18)	2,142 (0,59)	2,138 (0,72)
`nanochat`	1,969	1,970 (8,36)	1,958 (4,36)	2,072 (1,95)	2,062 (1,36)	2,047 (3,18)	2,094 (2,36)	2,163 (1,18)	2,128 (1,95)
`rnn`	1,964	1,964 (8,07)	1,972 (4,07)	2,065 (1,65)	2,076 (1,07)	1,990 (2,17)	2,040 (1,24)	2,108 (0,62)	2,102 (0,76)



Şekil 3. Kalite–bit dengesi iki model için. GPT-2'de (dar katmanlar, $C = 32$) binary, LittleBit'ten aynı bitte daha iyi. makemore MLP'de (ilk katmanın girişi 256) LittleBit, binary'den daha az bitle daha iyi.

- **INT4** her modelde float'la aynı; KD sayesinde bazen biraz daha iyi.
- **Ternary** binary'den fazla bit harcıyor, sonucu binary'den iyi değil.
- **Binary** transformer'larda +0,03…0,09 nat: en iyi 1 bitlik seçenek.
- **LittleBit** dar katmanlarda binary'nin gerisinde, girişi geniş katmanlarda önde. Dual-SVID başlangıcı tek başına yetmiyor (eğitimsiz kayıp 3–12 nat); QAT şart.

Neden? Scale'lerin bedeli. §5'teki BPW formülünün ikinci terimi $(16(d_o + d_i + r)/(d_o d_i))$ scale vektörlerinin bedelidir. Binary bir katmanın bedeli ise $1 + 16/d_i$'dir. Sayısal örnek: 32×32'lik bir katmanda yalnız scale'ler $(r = 0)$ weight başına $16 \cdot 64/1024 = 1$ bit tutar; bu, binary katmanın toplam bedelinin (1,5) üçte ikisi. 4096 × 4096'lık bir Llama 2 katmanında aynı terim 0,008'e iner. LittleBit'in 0,1 BPW sonucu bu ikinci dünyaya aittir; 32 genişliğindeki bir modelde 0,1 BPW matematiksel olarak mümkün değil.

Sonuçlar: stories260K. Karpathy'nin hazır modeli (float 1,268) çok daha zor bir sınav oldu (Tablo 8).

Tablo 8. stories260K, 100 test penceresi × 256 token.

Yöntem	BPW	Eğitimsiz	QAT
`int8`	8,21	1,269	–
`int4`	4,21	1,428	1,311 (2 000 adım)
`bin`	1,21	7,281	1,933 (3 000 adım)
`lb25p2`	1,63	12,581	2,743 (300 adım)
`lb25p1`	0,82	17,861	2,486 (3 000 adım)

Karar: donanıma 1 bitlik weight. INT4'ün bu ölçekteki tek kazancı belleği yarıya indirmek ve modellerimiz BRAM'e zaten INT8'le sığıyor. Ternary binary'den iyi değil. Binary ise weight'i 8 kat küçültür *ve* matris-vektör çarpımını çarpıcısız toplama-çıkarmaya çevirir. LittleBit'in faktörleri de binary matrisler olduğu için aynı birimi kullanabilir. Engine'e bu yüzden iki şey eklendi:

- **GEMVB:** `wbin.mem`'deki 64 bitlik bir kelime, bir girişin 64 çıkış için işaretlerini taşır ($1 = +1$, $0 = -1$). 64 toplama/çıkarma hattı vardır, DSP kullanılmaz. Çıkış başına scale, requant'ın m_0 çarpanına katlanır. Akümülatör 20 bittir, çünkü $|acc| \le 128\, n_{\text{in}} < 2^{19}$.
- **SCALE:** girişe kanal başına INT16 bir gain uygular. Derleyici normalde norm gain'lerini γ sonraki weight'e katlar $(W\text{diag}(\gamma))$ ve bu binary weight'i bozar. `ir.factor_binary` katlanmış matrisi $\text{diag}(a)\, S\, \text{diag}(g)$ olarak ayırır: a requant'a, g SCALE'e gider, $S \in \{\pm 1\}$ GEMVB'ye.

Tek yollu LittleBit için yeni donanım gerekmedi: `ir.factor_littlebit` her katmanı $\text{SCALE}(g \cdot \gamma) \to \text{GEMVB}(\text{sign}(V)^\top) \to \text{GEMVB}(\text{sign}(U))$ dizisine çevirir; faktörlere ayrılmış program, yoğun weight'li modelle aynı logit'leri verir (fark $\le 2 \cdot 10^{-6}$).

Tablo 9'e göre binary GPT-2 1,75 kat hızlı ve +0,03 nat; LittleBit MLP 0,59 BPW'de 3,6 kat hızlı ve +0,16 nat; binary stories260K 1,8 kat hızlı, BRAM %72'den %41'e iniyor, ama +0,64 nat kaybediyor.

Tablo 9. Düşük bitli modeller kart tasarımında (Yosys kaynakları, kart simülasyonundan cycle, kartın bit-exact çalıştırdığı integer modelin test kaybı).

Model	Weight	Kayıp	cycle/token	LUT	FF	BRAM36	DSP
`gpt2`	INT8	2,002	8 712	7 112	3 082	19,5	58
`gpt2-bin`	binary (Larq tarzı)	2,028	4 983	11 153	5 778	10,0	60
`mlp`	INT8	1,987	6 722	4 949	2 167	14,0	22
`mlp-lb25p1`	LittleBit 0,59 BPW	2,142	1 874	7 940	4 767	5,0	25
`stories260k`	INT8	1,291	50 397	9 002	3 312	97,5	66
`stories260k-bin`	binary (Larq tarzı)	1,927	27 992	12 118	5 920	56,0	68

Sorunlar ve çözümleri

- **GEMVB çok LUT tutuyordu:** ilk sürüm 14 709 LUT'tu. 64 hattın akümülatörleri INT8 GEMV'nin geniş bankasına eklenince her şey büyüyordu → ayrı, 20 bitlik bir akümülatör/shadow bankası; INT8 akümülatör genişliği eski hâline döndü: 11 153 LUT. Banka yalnızca modelde GEMVB varsa sentezlenir.
- **Gereksiz dolgu:** binary katmanların çıkış parametreleri 64'e dolduruluyordu → GEMVB yalnız gerçek çıkışları boşaltıyor.
- **Eğitilmiş bir modelde öğrenme oranı:** stories260K'da INT4 QAT 5e-4 ile modeli bozuyordu; 1e-4 ile iyileştirdi (1,311). Binary 3 000 adımda lr 1,5e-3 ile 1,933'e indi; kayıp son adımlarda hâlâ düşüyordu (2,10 → 1,93).
- **İki yollu LittleBit:** iki yolun çıkışlarını bir activation'dan önce toplamak gerekiyor; şimdilik yalnız tek yollu LittleBit karta gidiyor.

Sunumdaki üç araştırma sorusunun cevabı. 16 Ekim'deki sunumun son slaytı üç soru soruyordu; bu evre üçünü de cevapladı:

1. *Küçük modellerde LittleBit'in anlamlı BPW alt sınırı nedir?* Scale'lerin tabanı 32 genişliğinde 1 BPW, 64'te 0,5 BPW. Ölçülen en düşük anlamlı nokta MLP'de 0,59 BPW, +0,16 nat.
2. *Binary matris-vektör birimi kaç LUT tüketir, token başına kaç cycle sürer?* SCALE dahil yaklaşık +4 000 LUT ve +2 700 FF, DSP yok; GPT-2'de 8 713 → 4 983 cycle/token.
3. *FP16 scale'ler integer'a indirilip bit-exact bir donanım kurulabilir mi?* Evet: çıkış başına scale'ler requant'ın 16 bitlik m_0'ına, giriş başına gain'ler INT16 SCALE'e gider; üç düşük bitli modelin üçü de kart tasarımında bit-exact.

Çıkan ders. Bir makalenin başlık sonucu (0,1 BPW) her ölçeğe taşınmaz; formül, nerede taşındığını söyler. Ölçümler formülle örtüştü: dar katmanlarda binary, geniş katmanlarda LittleBit önde. Donanım tarafında doğru soyutlama (1 bitlik GEMV + SCALE) iki yöntemi de ek birim olmadan çalıştırdı.

8 Evre 7: Doğrulamanın sertleştirilmesi

27 Ekim 02:03 – 04:00 · commit'ler `9f8ee72`, `543c443`, `85ce47d`, `4e0782a`

Neden?

"Bütün modeller bit-exact" cümlesi, testin neyi denediği kadar güçlüdür. O ana kadarki kart testi her modelde aynı birkaç senaryoyu çalıştırıyordu: otomatik ilk koşu, BTNC, greedy ve temizleme. Üç soru açıktaydı:

- Bir kullanıcı koşu *sırasında* düğmeye basarsa ya da reset verirse ne olur?
- Simülatörün başlattığı bütün değerler 0'dır; gerçek çipte ilk değeri verilmemiş bir register ne yapar?
- RTL doğru olsa bile, onu kapılara çeviren sentez aracı doğru mu?

Bu iş, düşük bit çalışmasıyla eş zamanlı olarak ayrı bir dalda bir ajan tarafından yapıldı ve sonra ana dala birleştirildi.

Ne yapıldı?

- **Testbench yeniden yazıldı:** koşu sırasında tuşlar ve reset; girişler düşen kenarda sürülüyor, çıkışlar düşen kenarda örnekleniyor; her cycle'da bütün pinlerin bir imzası; 7-segment okunuyor; VGA karesi pinlerden alınıp senkron sinyalleri kontrol ediliyor.
- **Rastgele testler** (`board_fuzz.py`): rastgele anahtar ayarları ve tuş dizileri; 7 modelde toplam 732 koşu, 135 180 token. Her koşuda Python'daki `Board` modeli, aynı tuş dizisinin *ne yapması gerektiğini* hesaplıyor (kuyruğa alınan koşular, reset, temizleme).
- **Yönlendirilmiş köşe durumları:** 16 durum. Örnekler: koşu sırasında iki kez BTNC (tek bir kuyruklu koşu olmalı), son token'dan hemen önce reset, daktilo modunun token'ları ve sayılan cycle'ları değiştirmemesi.
- **4-state simülasyon** (sv2v + Icarus): 0 ve 1'in yanında X (bilinmeyen) değerini de taşıyan simülasyon. gru'da, koşu ortasında reset dahil 2,7 milyon cycle; pinlerde ve metin RAM'inin yazma portunda hiç X yok.
- **Sentez sonrası netlist simülasyonu:** gerçek üst modülün (MMCM, BUFG, BRAM dahil) Yosys netlist'i Verilator'da çalıştırılıp RTL ile *her pinde, her cycle'da* karşılaştırıldı. gru'da 5 040 047 cycle boyunca aynı. Bunun için iki model yazıldı: Yosys'in hücre kütüphanesindeki BRAM modellerinin davranışı yok (`tb/sim_ramb.v`), MMCM için de bir geçiş modeli (`tb/sim_mmcme2_base.v`). 13 Ekim'de sonuçsuz kalan netlist simülasyonu (§4) böylece çalıştı.

Bulunan hatalar

Testler altı sorun buldu (Tablo 10). Üçü bizim kodumuzdaydı, üçü kullandığımız araçlarda.

Tablo 10. Doğrulamanın sertleştirilmesiyle bulunan sorunlar.

Sorun	Nerede	Belirti ve çözüm
BTNU kayboluyordu	RTL (`lm_gen`)	Kuyruktaki bir koşunun başladığı cycle'a denk gelen BTNU basışı ne yeniden tohumluyor ne de ek koşu başlatıyordu. Artık bir sonraki koşu olarak kuyruğa alınıyor; bir yarış testiyle doğrulandı.
Heartbeat reset'siz	RTL (top)	4-state simülasyonda LED0 ve LED16_B sürekli X'ti. Sayaç `rst_n` ile sıfırlanıyor.
İyimser token/s	RTL (`lm_gen`)	Bigram'da ekranı bekleme cycle'ları sayılmıyordu: 126 değil 157 cycle/token. Diğer modellerde fark yok.
sv2v işaret kaybı	sv2v 0.0.13	Signed bir dizi elemanının boyut cast'i (`64'(att_a[j])`) işaretsiz çevriliyordu; IEEE 1800 §6.24.1'e göre cast işareti korur. Attention çıkışındaki `>>> 32` mantıksal kaydırmaya dönüşüp negatif değerleri +127'ye doyuruyordu. Açık `$signed` yazıldı.
Yosys optimizasyon hatası	Yosys 0.69	`wreduce` ve ardından `peepopt`, q·k toplamındaki dinamik bayt seçimini yanlış optimize ediyordu (iki adım tek başına doğru). RTL önce head dilimini seçip sabit indeks kullanacak şekilde yeniden yazıldı.
Yosys DSP eşlemesi	Yosys 0.69	Requant çarpımı DSP48E1'e eşlenince (girişler register'dan geliyorsa) işaret uzatması düşüyor, negatif akümülatörler +max'a doyuyor. RTL doğru; netlist akışı çarpıcıları LUT'a eşliyor (`-nodsp`); `requant_netlist_check.sh` hatayı tek başına gösteriyor.

İki araç hatasının hikâyesi. sv2v ve Yosys hataları ilginçti, çünkü *eski RTL'de de vardı* ve 13 Ekim'den beri fark edilmemişti. Verilator ve Vivado SystemVerilog'u doğrudan okuduğu için doğru sonuç veriyordu; yalnız sv2v'den geçen akışlar (Icarus ve Yosys netlist'i) bozuktu. Yosys hatasının belirtisi yanıltıcıydı: $t = 0$'da attention çıkışı skordan bağımsız olduğu için ilk pozisyonlar doğru, sonrakiler yanlıştı. İlk hipotez (çıkarmalı bir kaydırma) yanlış çıktı ve geri alındı; hata, Yosys'in adımları tek tek açılıp kapatılarak iki adımın birleşimine kadar daraltıldı. Düzeltmelerden sonra:

- Icarus 4-state'te gpt2, llama, nanochat, microgpt, bow ve gpt2-bin bit-exact;
- engine'in tam Xilinx sentezli netlist'i gpt2, llama, nanochat ve gpt2-bin'de bit-exact;
- gpt2-bin'in kart netlist'i 4 200 047 cycle boyunca her pinde RTL ile aynı (pin imzası `3adc10ab`); VGA karesinde 0 piksel fark.

Verilator'daki pin imzaları düzeltmelerden sonra değişmedi; yani RTL'in anlamı aynı kaldı, yalnız araçların onu yanlış anlamasının yolu kapandı.

Çıkan ders. Simülatör ve sentez aracı da yazılımdır ve hata yapar. Aynı RTL'i birbirinden bağımsız araç zincirlerinden (Verilator; sv2v + Icarus; sv2v + Yosys) geçirmek, tek bir zincirin göremeyeceği hataları ortaya çıkardı. İkinci ders: testin kendisi bir modeldir; Python'daki `Board` modeli, düğmelerin *ne yapması gerektiğini* yazılı hâle getirdiği için iki gerçek RTL hatasını yakalayabildi.

9 Evre 8: Mac'te açık kaynak yerleşim, yönlendirme ve zamanlama

27 Ekim 05:55 – 10:54 · commit'ler `196d9fc`, `502e744`, `f2c5ff8`, `0883450`, `bb001a0`

Neden?

13 Ekim'den beri bir soru cevapsızdı: devre çipte *kaç MHz'te* çalışır? Sentez, devreyi LUT, flip-flop, BRAM ve DSP'lerden oluşan bir listeye çevirir ama bu parçaların çipte nereye konacağını ve aralarındaki kabloların ne kadar gecikme getireceğini söylemez. Bunu yerleşim ve yönlendirme (place and route, P&R) ve ardından statik zamanlama analizi (STA) söyler. Vivado bu işi yapar ama Mac'te çalışmaz. İki yol vardı: Vivado'lu bir bilgisayar ya da Mac'te açık kaynak araç zinciri (nextpnr-himbaechel ve Project X-Ray). Bu sırada sunumdaki FPGA slaytları da Nexys A7-100T'nin sayılarına göre güncellendi (`196d9fc`).

Ne yapıldı?

Araç zinciri ve izin denetimi. Açık kaynak tarif (`xc7-toolchain`), nextpnr'ı ve Project X-Ray'i kaynak koddan derlemeyi gerektiriyor. Bağımlılıklar Homebrew'den kuruldu, kaynaklar ve 420 MB'lık çip veritabanı `~/tools/xc7` altına indirildi. Ama derleme adımı, oturumun otomatik izin denetiminde "dışarıdan gelen kodu çalıştırmak" sayılarak iki kez reddedildi; ikincisi kullanıcının "devam et" demesinden sonraydı. Denetim dolanılmadı. Bunun yerine derleme ve akış betikleri yazıldı (`scripts/xc7_toolchain.sh`, `scripts/nexys_pnr.sh`, `502e744`) ve derlemeyi kullanıcı kendisi başlattı:

> *"! make xc7-toolchain"* 27 Ekim, 10:29

İlk deneme: macOS SDK. Derleme, araç zincirinin koduna gelmeden durdu: `xcrun` en yeni Command Line Tools SDK'sını (MacOSX27.0) seçiyordu, Xcode'un linker'ı ise bu SDK'daki yeni mimari adını tanımıyordu (`unknown architecture arm64e.x1`). Bu, 26 Ekim'de simülasyon betiklerinde çözülen sorunun aynısıydı; aynı `SDKROOT` düzeltmesi betiğe eklendi (`f2c5ff8`). İkinci denemede derleme birkaç dakikada bitti: `nextpnr-himbaechel`, 39 MB'lık xc7a100t çip veritabanı, `xc7frames2bit` ve `fasm2frames`.

İkinci engel: nextpnr'da bir hata. İlk yerleşim denemesi `Unable to constrain IO 'LED16_G', device does not have a pin named ''` hatasıyla durdu. XDC dosyasında pin açıkça yazılıydı (M16). Hata nextpnr'ın kendisindeydi: `attrs[LOC] = attrs.at(PACKAGE_PIN)` satırında, sağdaki referans alınır, soldaki `operator[]` yeni bir eleman ekleyip tabloyu yeniden ayırır ve kopyalama boşalmış bir referanstan yapılır; pin adı boş çıkar. Akış, pinleri doğrudan `LOC` ile yazarak bu yolu atladı (`0883450`).

İlk sonuç. bigram uçtan uca geçti: sentez 18 s, yerleşim/yönlendirme 21 s, bitstream 19 s. Fmax 71,3 MHz, yani 50 MHz hedefini rahat tutuyordu; bitstream 3 825 962 bayt.

Sezgi ve sayısal örnek: Fmax. Senkron bir devrede her clock kenarında flip-flop'lar yeni değerlerini alır. Bir flip-flop'tan çıkan sinyal, aradaki mantıktan ve kablolardan geçip bir sonraki flip-flop'a *bir sonraki kenardan önce* ulaşmalıdır:

$$T_{\text{clk}} \;\geq\; t_{\text{clk}\to\text{q}} + \sum_{\text{yol}} \left(t_{\text{mantık}} + t_{\text{kablo}}\right) + t_{\text{setup}}, \qquad f_{\max} = \frac{1}{\max_{\text{yol}}\left(\dots\right)}.$$

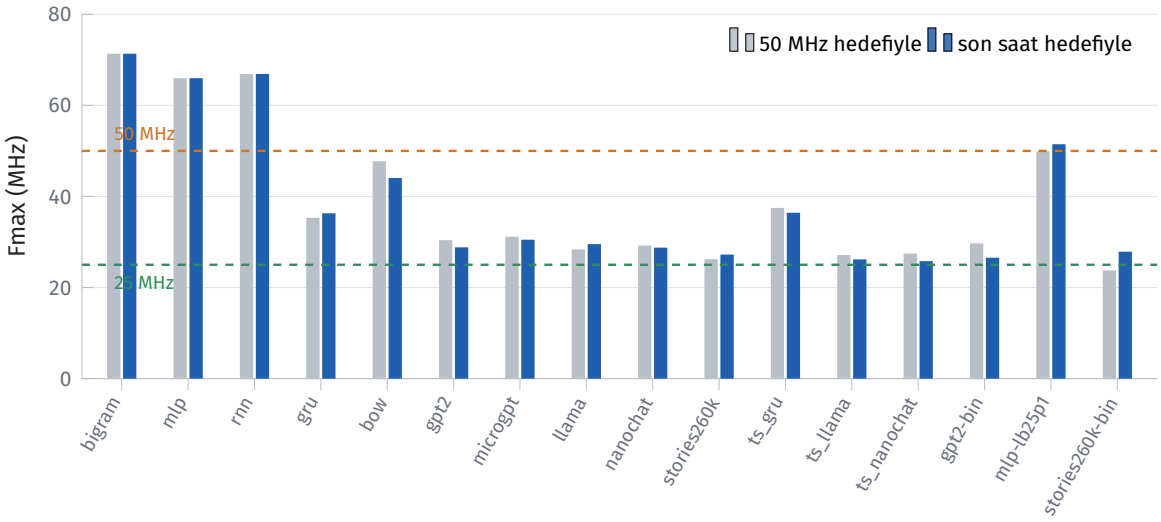
50 MHz'in periyodu 20 ns'dir. gpt2-bin'in en uzun yolu 33,67 ns sürüyor; 1/33,67 ns ≈ 29,7 MHz. Bu yol 20 ns'ye sığmaz ama 25 MHz'in 40 ns'lik periyoduna sığar (Şekil 5).

Toplu koşu ve 25 MHz'e düşüş. 18 modelin ilk toplu koşusunda bow (48,2 MHz), gru (35,3 MHz) ve gpt2-bin (29,7 MHz) 50 MHz'i tutmadı. nextpnr zamanlama tutmayınca hata koduyla çıktığı için bunlar ilk anda "hata" gibi göründü. Akış, Vivado betiğiyle aynı kurala getirildi: 50 MHz tutmazsa model 25 MHz için yeniden kurulur; bitstream yalnız zamanlama tutunca yazılır (`bb001a0`). Çekirdek clock 100 MHz'lik osilatörden MMCM ile üretildiği ve VGA'nın piksel clock'u bundan bölündüğü için seçilebilen değerler 25, 50 ve 100 MHz'dir. Tablo 11 ve Şekil 4 sonuçları gösterir.

Tablo 11. Açık kaynak akışla yerleşim/yönlendirme sonuçları (nextpnr-himbaechel, xc7a100tcsg324-1, çarpıcılar LUT'ta). "Pay", son saatte Fmax'ın hedefi ne kadar aştığıdır. LUT sütunu nextpnr'ın LUT bel sayısıdır; çipte 126 800 bel vardır (her fiziksel LUT6 iki bel). BRAM, RAMB36 eşdeğeri.

Model	50 MHz'de Fmax	Saat	Fmax	Pay	Zamanlama	LUT bel	FF	BRAM
`bigram`	71,3	50	71,3	43 %	tuttu	6 362	1 334	2,0
`mlp`	66,0	50	66,0	32 %	tuttu	13 631	2 228	14,0
`rnn`	66,9	50	66,9	34 %	tuttu	12 705	2 194	13,5
`gru`	35,3	25	36,3	45 %	tuttu	15 506	2 255	16,0
`bow`	47,7	25	44,0	76 %	tuttu	15 234	2 222	15,0
`gpt2`	30,4	25	28,8	15 %	tuttu	29 435	3 265	19,5
`microgpt`	31,2	25	30,5	22 %	tuttu	29 977	3 277	18,0
`llama`	28,4	25	29,5	18 %	tuttu	33 274	3 439	17,5
`nanochat`	29,2	25	28,8	15 %	tuttu	32 288	3 416	16,0
`stories260k`	26,2	25	27,2	9 %	tuttu	34 643	3 513	97,5
`ts_gpt2`	yerleşim takıldı: 149 RAMB18 + 41 RAMB36 ≈ 115,5/135 BRAM yeri							
`ts_gru`	37,5	25	36,4	46 %	tuttu	15 826	2 289	69,5
`ts_llama`	27,1	25	26,2	5 %	tuttu	34 612	3 504	97,5
`ts_microgpt`	yerleşim takıldı: 140 RAMB18 + 43 RAMB36 ≈ 113,0/135 BRAM yeri							
`ts_nanochat`	27,5	25	25,8	3 %	tuttu	34 550	3 470	90,5
`gpt2-bin`	29,7	25	26,6	6 %	tuttu	35 041	5 958	10,0
`mlp-lb25p1`	49,9	25	51,5	106 %	tuttu	18 925	4 822	5,0
`stories260k-bin`	23,8	25	27,9	11 %	tuttu	39 334	6 120	56,0

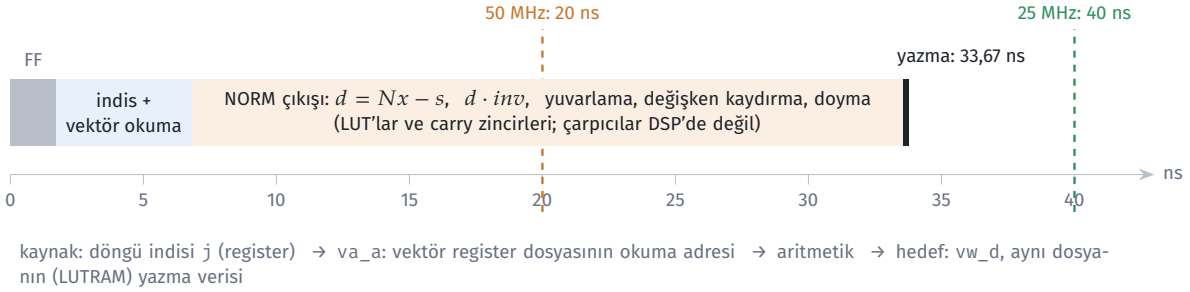
Neden bu kadar yavaş? Kritik yol. nextpnr'ın raporundaki en uzun yol, gpt2-bin'de döngü indisi `j`'den başlıyor, vektör register dosyasının okuma adresinden (`va_a`) geçiyor ve aynı dosyanın yazma verisinde (`vw_d`) bitiyor. RTL'de bu uç noktaların karşılığı NORM biriminin çıkış aşamasıdır: okunan eleman x'ten $d = Nx - s$, sonra $d \cdot inv$, yuvarlama, değişken bir kaydırma ve



Şekil 4. Fmax, model başına. Gri: 50 MHz hedefiyle yapılan ilk deneme; mavi: son saat hedefiyle. Model büyüdükçe Fmax 25–30 MHz bandına iniyor.

8 bite doyma, hepsi *tek bir cycle'da*. Transformer'larda (stories260K, llama, nanochat) yol aynı kalıpta ama attention'ın head dilimi qh'den geçiyor: stories260K'da 36,7 ns, bunun 9,7 ns'si mantık, 26,9 ns'si (%73) kablolama. Yolun yeri modele göre değişse de kalıp aynı: tampondan oku, çarp, tampona yaz. İki etken bu yolu uzatıyor:

- **Tek cycle'lık eleman bazlı yol:** engine, okuma–hesap–yazma işini aynı cycle'da yapacak şekilde tasarlandı. Simülasyonda bunun bir bedeli yoktu, zamanlamada var.
- **Çarpıcılar LUT'ta:** Yosys'in DSP hatası (§8) yüzünden bit-exact olduğu kanıtlanan netlist çarpıcıları LUT'larla kuruyor. Bir DSP48E1 çarpması birkaç ns sürer; LUT'larla kurulmuş geniş bir çarpma çok daha uzun bir carry zinciridir. Bunun kaynak bedeli de büyük: stories260K'da Yosys'in DSP'li tahmini 9 002 LUT ve 66 DSP iken LUT'a eşlenmiş netlist 34 643 LUT bel'i tutuyor.



Şekil 5. gpt2-bin'in kritik yolu, birikimli gecikmeyle (nextpnr, 50 MHz hedefi). Yol 20 ns'lik periyoda sığmıyor, 40 ns'lik periyoda sığıyor.

Önerilen çözüm (henüz yapılmadı). Yolun ortasına bir register koymak (pipeline): okuma ve d bir cycle'da, çarpma–kaydırma–doyma bir sonraki cycle'da. Her iki yarı da 20 ns'nin altına inerse model 50 MHz'te çalışır; bedeli, bu işlemlerin bir cycle geç bitmesidir. Bu bir RTL değişikliği olduğu için 18 modelin bütün testlerinin yeniden geçmesi gerekir.

Sorunlar ve çözümleri

- **Üçüncü taraf kodu derleme izni** → betikler hazırlandı, derlemeyi kullanıcı başlattı.
- **macOS SDK / linker uyumsuzluğu** → `SDKROOT`, Xcode'un SDK'sına.
- **nextpnr'da boşalmış referans** → pinler `LOC` ile.
- **Zamanlama tutmayınca nextpnr'ın hata vermesi** → `--timing-allow-fail` ile rapor her zaman yazılıyor, karar betikte veriliyor.

Çıkan ders. Bir sayının "ölçülmedi" olarak kalması, tasarımın bir zayıflığını gizleyebilir. Fmax ölçülünce engine'in tek cycle'lık yapısının bedeli ortaya çıktı: büyük modellerde 25 MHz. Bu, cycle/token'ın tek başına hız ölçüsü olmadığını hatırlattı: hız = clock / (cycle/token).

10 Evre 9: Karar, fiziksel kart yok

27 Ekim 10:40 – 11:49

Neden?

Toplu yerleşim koşusu sürerken kullanıcı, projenin ne yaptığını ve nasıl ölçtüğünü adım adım sordu: kaç model implemente ediliyor, nasıl bir ölçüm yapılıyor, modelin doğru implemente edildiği nasıl test ediliyor, bitstream nedir. Bu soruların sonunda proje için belirleyici bir karar geldi:

> *"karta yükleme yapmıycaz simüle edicez macte"* 27 Ekim, 11:49

Böylece doğrulama zincirinin son halkası, yani fiziksel kartta LED16'nın yeşil yanması, kapsam dışına çıktı. Soru şuna dönüştü: *kart olmadan, devrenin çipte doğru çalışacağını nasıl gösteririz?*

Ne yapıldı?

Kart yoksa ne yeterli? Kart üretilmeden önce endüstride yapılan doğrulama iki ayağa dayanır: devrenin mantıksal olarak doğru olduğunu gösteren simülasyon ve her sinyalin zamanında yerine oturduğunu gösteren statik zamanlama analizi. Bizim tasarımımızda bu ikisinin birlikte yeterli olmasının sebebi, tasarımın tek bir clock domain'inde çalışan senkron bir devre olmasıdır: girişler (düğmeler ve anahtarlar) senkronlanıp debounce edilir, çıkışlar ise insan gözüyle okunan LED'ler ve VGA'dır. Şekil 6, zincirin her halkasını ve neyi yakaladığını gösterir.



Şekil 6. Doğrulama zinciri. Her halka bir öncekine karşı kontrol edilir. 0–2 kalite ve mimari, 3–6 mantıksal doğruluk, 7 zamanlama; 8. halka (kart) kullanıcının kararıyla yapılmayacak.

Örnek: stories260K. RTL simülasyonu, kartın ürettiği her token'ın ve ekrandaki her karakterin Python referansıyla aynı olduğunu gösteriyor. Zamanlama analizi, 25 MHz'te en uzun yolun 40 ns'lik periyoda sığdığını söylüyor. İkisi birlikte: çip 25 MHz'te çalıştırılırsa, simülasyonda gördüğümüz hikâyenin aynısını, token başına 50 397 cycle ile, yani $25{\cdot}10^6/50\,397 \approx 496$ token/s hızla yazar.

Açık kalan iki boşluk.

- **Simüle edilen netlist, zamanlaması ölçülen netlist değil.** Netlist simülasyonu (§8) kısa debounce süreleriyle ve 50 MHz ayarıyla sentezlenmiş bir netlist'te yapıldı; yerleştirilen netlist ise 25 MHz ayarlı ve gerçek sürelerle sentezleniyor. Mantık aynı, sabitler farklı. Boşluğu kapatmak için akışın yerleştirdiği netlist'i Verilog olarak da kaydetmesi ve o netlist'in 18 modelde simüle edilmesi planlandı.
- **Yönlendirilmiş devre simüle edilmiyor.** nextpnr'ın bu sürümü, yerleşimden sonraki devreyi simüle edilebilir Verilog olarak yazamıyor. Bu son adımda zamanlama analizine güveniliyor. nextpnr'ın gecikme modeli topluluk tarafından çıkarılmıştır (Project X-Ray); Vivado'nun sonucu biraz farklı olabilir.

Çıkan ders. Kullanıcının soruları ("nasıl test ediyoruz?", "bitstream nedir?") aslında bir gözden geçirmeydi: her cevap, zincirin hangi halkasının neyi garanti ettiğini açıkça yazmayı gerektirdi. Kart kararı bir eksiklik değil, bir netleşmedir: iddia artık "kartta çalıştı" değil, "simülasyon ve zamanlama analiziyle çalışacağı gösterildi"dir ve bu iddianın sınırları bellidir.

KISIM IV

Bugün ve Sonrası

11 Bugünkü durum

27 Ekim 2026, bu raporun yazıldığı an

11.1 Kart tasarımındaki modeller

Tablo 12, kart tasarımına giren 18 modelin bugünkü durumunu tek yerde toplar. "Saat" sütunu, açık kaynak akışın zamanlamayı tutturduğu çekirdek clock'tur; token/s bu saatten hesaplanır. Henüz yerleştirilmemiş modeller "sürüyor" olarak görünür; tablo `veri.py` yeniden çalıştırılınca güncellenir.

11.2 Ne bitti, ne kaldı?

11.3 Sonraki adımlar

1. **Engine'i pipeline'a bölmek, 50 MHz:** NORM çıkışı gibi tek cycle'lık eleman bazlı yollara bir register eklemek (§9). Bedeli birkaç cycle; kazancı büyük modellerde yaklaşık iki kat token/s. Bütün testlerin yeniden geçmesi gerekir.
2. **Yerleştirilen netlist'in simülasyonu, 18 model:** akışın yerleştirdiği netlist'i Verilog olarak kaydedip gerçek clock ayarı ve gerçek debounce süreleriyle tam kart simülasyonundan geçirmek (§10).
3. **minGPT'nin sıralama modeli, 19. model:** projenin başladığı `gpt-nano` sıralama görevi bugün kart tasarımında yok; o model eski PYNQ hızlandırıcısında çalışıyordu. Zoo'daki `gpt2`

Tablo 12. 18 model, Nexys A7-100T kart tasarımında. cycle/token kart simülasyonundan (otomatik ilk koşunun ortalaması); "Sim." tam kart simülasyonunun Python referansıyla karşılaştırması.

Model	Veri	Weight	cycle/token	token/s, 50 MHz	Saat	token/s, saatte	Sim.
`bigram`	names	INT8	157	318 098	50 MHz	318 066	PASS
`mlp`	names	INT8	6 722	7 438	50 MHz	7 438	PASS
`rnn`	names	INT8	6 464	7 735	50 MHz	7 735	PASS
`gru`	names	INT8	6 605	7 570	25 MHz	3 785	PASS
`bow`	names	INT8	6 368	7 852	25 MHz	3 926	PASS
`gpt2`	names	INT8	8 712	5 739	25 MHz	2 869	PASS
`microgpt`	names	INT8	8 746	5 717	25 MHz	2 858	PASS
`llama`	names	INT8	9 490	5 268	25 MHz	2 634	PASS
`nanochat`	names	INT8	10 813	4 624	25 MHz	2 312	PASS
`stories260k`	TinyStories	INT8	50 397	992	25 MHz	496	PASS
`ts_gpt2`	TinyStories	INT8	41 781	1 197	*sürüyor*	*sürüyor*	PASS
`ts_gru`	TinyStories	INT8	30 407	1 644	25 MHz	822	PASS
`ts_llama`	TinyStories	INT8	50 397	992	25 MHz	496	PASS
`ts_microgpt`	TinyStories	INT8	40 570	1 232	*sürüyor*	*sürüyor*	PASS
`ts_nanochat`	TinyStories	INT8	50 680	987	25 MHz	493	PASS
`gpt2-bin`	names	binary	4 983	10 034	25 MHz	5 017	PASS
`mlp-lb25p1`	names	LittleBit	1 874	26 681	25 MHz	13 340	PASS
`stories260k-bin`	TinyStories	binary	27 992	1 786	25 MHz	893	PASS

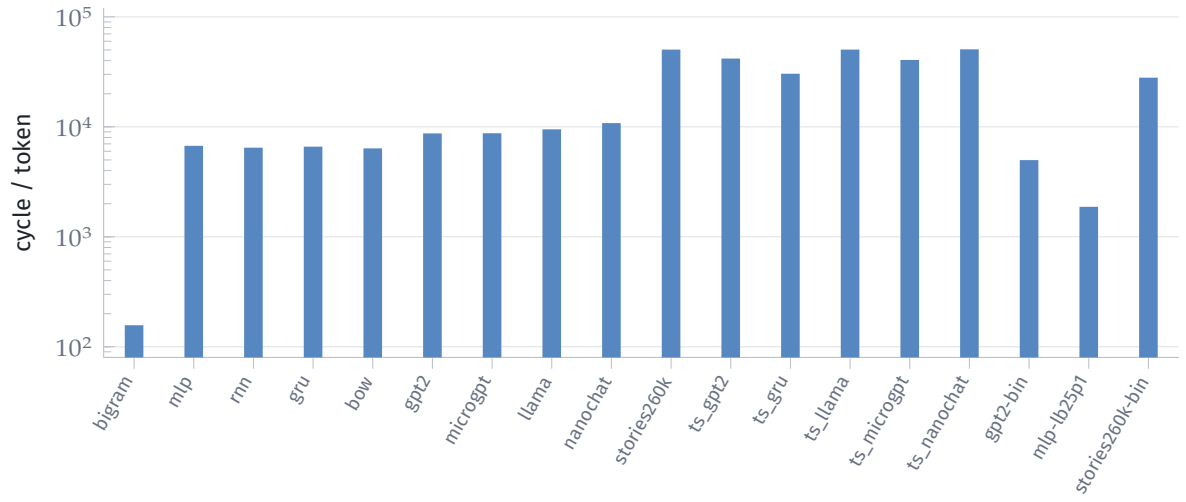
aynı GPT-2 bloğu ama names'te eğitildi. Ekrana girdi dizisini ve sıralanmış çıktıyı yazan bir görünümle eklenebilir. Sunumun konusu olduğu için önemli.

4. **Daha uzun binary QAT (stories260K) ve iki yollu LittleBit'in donanımda toplanması.**
5. **Yosys DSP hatasının** küçük örnekle (`requant_netlist_check.sh`) Yosys geliştiricilerine bildirilmesi; düzelirse DSP'li ve bit-exact bir açık kaynak bitstream.
6. **Sunumun geri kalanı:** 16 Ekim'deki sunum yalnız minGPT kısmını kapsıyordu; implementasyonu anlatan yeni bir sunum, bu raporla birlikte hazırlanıyor.

11.4 Geriye bakış

On beş günde projenin üç kez yön değiştirdiği görülüyor: tek modelden aileye, PYNQ'dan Nexys'e, bilgisayarın sürdüğü hızlandırıcıdan kartta kendi başına çalışan modele. Her yön değişikliği kullanıcıdan geldi ve her biri iddiayı daha dürüst hâle getirdi. Değişmeyen şey bit-exact kuralıydı; bu kural sayesinde her yeni katman (sampling, ekran, düşük bit, yeni araçlar) bir öncekinin doğruluğunu bozmadan eklenebildi. Üç genel ders:

- **Spesifikasyon çalıştırılabilir olmalı.** Python'daki integer model hem bir referans hem de bir test kâhinidir; RTL'deki her hata ve araçlardaki üç hata ona karşı bulundu.
- **Araçlar da hata yapar.** Aynı tasarımı birbirinden bağımsız zincirlerden geçirmek, tek bir zincirin göremeyeceğini gösterir.



Şekil 7. Token başına cycle (logaritmik eksen). names modelleri birkaç bin, TinyStories modelleri 30–50 bin cycle; düşük bitli modeller INT8 karşılıklarının altında.

Tablo 13. Projenin durumu.

İş	Durum	Not
Integer aritmetik, ispatlı sınırlar	bitti	LN, RMSNorm, RoPE, QK-norm, GRU durumları
9 mimari + stories260K, tek engine	bitti	PYNQ dönemi engine'i, $P = 8/16/32$
Kart tasarımı (Nexys A7-100T)	bitti	sampling, detokenizer, VGA, kart üstü kontrol
Düşük bitli weight'ler	bitti	GEMVB + SCALE; 3 model kartta
Tam kart simülasyonu	18/18	token, hash, ekran
Rastgele / köşe / 4-state	bitti	732 koşu, 16 durum
Sentez sonrası netlist simülasyonu	kısmi	gru, gpt2-bin, mlp-lb25p1 kart düzeyinde; 4 model engine düzeyinde
Yerleşim, yönlendirme, Fmax	16/18	büyük modeller 25 MHz'te
Fiziksel kart	—	kullanıcının kararıyla yapılmayacak

- **Ölçülmeyen sayı tasarımın zayıf yanını saklar.** Fmax ölçülünce tek cycle'lık yolların bedeli görüldü; sıradaki iş tam olarak bu.

A Commit geçmişi

Deponun bütün commit'leri, eskiden yeniye (`git log`'dan üretildi; saatler yerel saat).

Tarih	Commit	Başlık
13.10 00:30	`088c227`	minGPT INT8 inference accelerator: golden model, RTL, testbenches, docs
13.10 00:35	`4e77f63`	Add PYNQ-Z2 board support with on-chip self-test
13.10 09:59	`9fc2dc5`	Add LaTeX technical report (PDF) with TikZ diagrams
13.10 10:57	`768ff4e`	Report: Karpathy model family part and visual explanations

Tarih	Commit	Başlık
13.10 11:06	`2b50611`	Add open-source Yosys synthesis flow and results
13.10 12:05	`adf3075`	Karpathy model ailesi: 9 model, INT8 nicemleme, mikrokodlu ortak RTL
13.10 13:01	`2aef51f`	Model ailesi: stories260K, kalibrasyon incelemesi, sentez ve rapor Kısım IV
13.10 14:26	`68765c8`	Paper altyapısı: deney protokolü, doğrulama genişletmesi, C ISA simülatörü
13.10 14:46	`e484e0a`	Tek bitstream motoru, AXI4-Lite sistemi, PYNQ sürücüsü, Vivado betikleri, literatür
13.10 15:06	`9279312`	Makale taslağı, koşu pipeline'ı, P taraması, CI, kullanıcı notu
13.10 15:24	`d5a3ecc`	zoo_run.c: POSIX tanımı (glibc + -std=c99 için clock_gettime)
13.10 15:36	`233febf`	BoW karşılık tablosu iki 16 bitlik kelime: bütün tablolar 16 bit
13.10 15:41	`e35e9ab`	Makale: sonuç bölümü iskeleti, şekil şablonları, veri üreteci; hw_final.py
13.10 15:46	`ad7aa10`	Kapalı biçimli çevrim modeli (RTL ile çevrimi çevrimine aynı), PYNQ demo defteri
13.10 15:47	`6926748`	CI: zaman sınırları, AXI testi tek modele indirildi
13.10 16:07	`a9a7e72`	AXI tezgâhı: yarışsız (düşen kenar) BFM; netlist simülasyonu betikleri
13.10 16:10	`9c6616b`	Rapor: deney protokolü, doğrulama, sistem ve ilgili çalışmalar bölümleri (taslak)
13.10 16:11	`c117cb8`	finalize.sh ve watch.sh: kampanyalar bitince doğrulama, terfi, sentez ve PDF'lerin otomatik üretimi
13.10 16:55	`6bb4ae9`	P tasarım uzayı: P=8/16/32, 6 model, 18 noktanın hepsi bit-exact; çevrim modeli her P'de birebir
15.10 12:48	`08523c8`	Sunum: minGPT bölümü (28 slayt), konuşma metni ve ilk tam taslak
15.10 12:50	`0809c16`	Sunum: konuşma metninin PDF hâli
16.10 13:24	`686116f`	Sunum: embedding'in matematiği ve araştırma konusu LittleBit (41 slayt)
16.10 13:47	`53caa4d`	Sunum: embedding bölümü genişletildi, LittleBit sezgisel anlatımla yeniden kuruldu (52 slayt)
16.10 14:09	`f9b9055`	Sunum: LittleBit bölümünde "düğme" yerine teknik terim scale kullanıldı
16.10 14:21	`81b3641`	Sunum: LittleBit bölümüne terimler, quantization ve LoRA slaytları (55 slayt)
26.10 23:35	`d76121c`	Nexys A7-100T: model kartta bağımsız çalışıyor (BRAM'de weight, FPGA'de sampling, detokenizer, VGA)
27.10 00:44	`8db3890`	Düşük bitli weight'ler: Larq tarzı binary/ternary ve LittleBit QAT, engine'e 1-bit GEMV (GEMVB) ve SCALE
27.10 00:57	`fc6d07a`	Düşük bit çalışmasının sonuçları ve belgesi; binary stories260K kartta
27.10 01:01	`7059b2e`	Düşük bit belgesi: stories260K INT4 QAT 2000 adım (1,311)
27.10 01:26	`76fbd65`	Düşük bit belgesi: stories260K LittleBit tek yol 3000 adım (2,486, binary 1,933)
27.10 02:03	`9f8ee72`	Nexys A7: kart tasarımının doğrulaması güçlendirildi (fuzz, 4-state, netlist, köşe durumları)
27.10 02:10	`543c443`	Doğrulama dalı birleştirildi: fuzz, köşe durumları, 4-state, netlist; iki RTL hatası düzeltildi
27.10 02:50	`85ce47d`	zoo_core: attention çıkışında açık $signed (sv2v tabanlı akışlarda attention yanlıştı)
27.10 04:00	`4e0782a`	zoo_core: attention skorunda dinamik bayt seçimi kaldırıldı (Yosys peepopt hatası); netlist bit-exact
27.10 05:59	`196d9fc`	Sunum: FPGA sayıları Nexys A7-100T'ye göre (XC7A100T: 63 400 LUT, 126 800 FF, 135 BRAM ~607 KB, 240 DSP)
27.10 09:22	`502e744`	Nexys A7: Vivado'suz açık kaynak uygulama akışı (Yosys + nextpnr-himbaechel + Project X-Ray)
27.10 10:30	`f2c5ff8`	xc7_toolchain: macOS SDK düzeltmesi (CLT SDK'sı Xcode linker'ıyla okunamıyordu), yarım kalan build dizini temizlenir

Tarih	Commit	Başlık
27.10 10:38	`0883450`	nexys_pnr: pin kısıtı LOC ile (nextpnr PACKAGE_PIN'i boşalmış bir referanstan kopyalıyordu); bigram uçtan uca: Fmax 71,3 MHz, bitstream
27.10 10:54	`bb001a0`	nexys_pnr: 50 MHz tutmazsa 25 MHz'e iner (Vivado betiği gibi), bitstream yalnız zamanlama tutunca; sonuçlar model/zoo/nexys.json'a

B Araçlar ve sürümler

Tablo 15. Bu raporun sonuçlarını üreten araçlar (macOS 27.0.1, Apple Silicon).

Araç	Sürüm / görev
Verilator	5.052: RTL ve netlist simülasyonu (birincil)
Icarus Verilog	13.0: 4-state (X) simülasyon
sv2v	0.0.13: SystemVerilog → Verilog (Icarus ve Yosys için)
Yosys	0.69+post: `synth_xilinx`, kaynak tahmini ve netlist
nextpnr-himbaechel	`3e53a0bf` (xilinx, yalnız xc7a100t): yerleşim, yönlendirme, zamanlama
Project X-Ray	`c9f02d85`: çip veritabanı, `fasm2frames`, `xc7frames2bit`
openFPGALoader	1.2.0: kart programlama (kullanılmayacak)
PyTorch	eğitim ve QAT (CPU ve MPS)
XeLaTeX	bu rapor ve teknik rapor

C Nerede ne var?

Tablo 16. Raporun anlattığı işlerin depodaki yerleri.

Yol	İçerik
`sw/`, `rtl/gpt_top.sv`	Evre 1: gpt-nano, integer referans, ilk RTL
`report/rapor.pdf`	Evre 2–3: teknik rapor (matematik, ispatlar, aile)
`sw/zoo/`, `rtl/zoo/zoo_core.sv`	Evre 3, 6: aile, IR derleyicisi, quantization, engine, GEMVB/SCALE
`sw/c/zoo_run.c`	C ISA simülatörü
`board/pynq_z2/`	PYNQ dönemi: AXI, sürücü, demo defteri
`sunum/`	Evre 4: sunum, konuşma metni
`board/nexys_a7/`, `sw/zoo/board.py`	Evre 5: kart tasarımı ve referansı
`sw/zoo/lowbit.py`, `docs/06_dusuk_bit.md`	Evre 6: Larq ve LittleBit QAT
`tb/tb_nexys.sv`, `sw/zoo/board_sim.py`, `board_fuzz.py`	Evre 7: kart doğrulaması
`scripts/xc7_toolchain.sh`, `scripts/nexys_pnr.sh`	Evre 8: açık kaynak uygulama akışı
`model/zoo/*.json`	bütün sonuç sayıları (bu raporun tabloları)
`docs/SENIN_YAPACAKLARIN.md`	Mac'te yapılamayanlar için notlar